Official IAM Operations Manual: Instrument Diagnostics and Chain Decoder

The first commissioned IAM chain. A research instrument, not a diagnostic test. Generated 2026-10-20 by manual/build_manual_v4.py from the files named at the start of each part.

Part 1. The Chain Decoder

Source: `manual/OM_v4_decoder.md`

The Chain Decoder

The chain in numbers

Read from the code at build time: the one command can load **22 Python files** holding **260 functions** (nested ones included); 260 of them have their entry in this Decoder, and a push is refused when one does not. Called: how many of a file's functions each traced run used (the array run on a healthy whole blood; the sequencing run on the release check's planted BAM).

file	lines	functions	called, array run	called, sequencing run
`MethylPhys_Interface/run_sample.py`	702	13	11	10
`stage_0_intake.py`	1,132	38	33	1
`stage_0_1_qc_handoff.py`	180	7	7	0
`stage_1_idat_calibration.py`	232	6	6	0
`conductor_v3.py`	309	19	18	0
`stage_m_met_a.py`	81	6	3	0
`MethylPhys_Interface/report_v4.py`	258	20	17	17
`MethylPhys_Interface/report_v3.py`	297	9	9	9
`MethylPhys_Interface/report_figs_v4.py`	145	7	6	2
`dev_stages.py`	336	28	3	0
`stage_v4.py`	143	8	4	2
`stage_deconv_v21.py`	198	6	4	0
`stage_q2_selfref.py`	270	29	0	11
`stage_atlas_commission.py`	62	8	1	0
`MethylPhys_Interface/report_pdf_v4.py`	100	6	6	6
`dev_comp_methods.py`	216	17	0	0
`deconv_v2.py`	74	3	0	0
`sky_statistics.py`	108	6	0	0
`stage_q0_intake.py`	127	5	0	0
`stage_q_iam_a.py`	174	9	0	0
`serial_mode.py`	67	6	0	0
`stage_groupa.py`	100	4	0	0

How to read the Decoder

The chain is the code that turns one sample into one report. This part of the manual takes it apart in the order a real run executes it: a traced run of the one command on a real healthy whole-blood array (GSE235717, GSM7508429; `chain/trace/`), and a second traced run on a small constructed sequencing file for IAM-A. Every function the one command can load has its own entry, in the file where it lives, in the order the run first enters that file.

Each stage opens with what it is for, the physics in words and then the formula, what it decides, every gate with its measured value, its limit, where the limit came from and whether it is provisional, the files it reads, and how to break it. Each function entry then says what the function does and what it decides. Under every entry the manual prints, read from the code itself and checked at every push: the file and line, the arguments, the module constants it reads, the data files it names, and how many times each traced run called it.

Every stage carries one of two labels, so the physics can be told apart from the bookkeeping:

- **Standard method**: a technique used elsewhere in the field (array calibration, deconvolution of a cell mix, normalization

against references run the same way). IAM uses it as a tool. It is not evidence for IAM's law, and a reviewer can check it against the literature.

- **From IAM's physics**: a quantity that follows from the cost of holding a mark against thermal noise. The entry gives the

derivation and the test that could have failed.

Names. The gauge has three marks. The **floor** is the least entropy a held mark can carry, derived from thermal kicks against about one ATP per site (M = 20.94). The **ceiling** (saturation) is every identity site fully scrambled. The **middle**, healthy = 1, is the healthy reference: Met-A's is measured once on purified healthy cells of the same type, and IAM-A's is computed from the sample's own loose share. Some file names in the chain predate these names: `Runtime Matrices/Met_A_Floors/metA_floors_v1_3.json` holds Met-A's healthy middle reference, not the floor. The Decoder uses the canon's names and says which file holds what.

Built from what, tested on what. A reviewer's first question about any reference is whether it was tested on the data it was built from. Every reference entry names both.

1. The one command: `run_sample.py`

What it is for. One sample in, one report out. A researcher runs one command on an IDAT pair (an Illumina EPIC v1 array), a calibrated beta table, or sequencing data (a wgbstools `.pat` file or an aligned bisulfite BAM). `run_sample.py` decides which stages run, in what order, and stops the run the moment a stage refuses the sample. It computes no reading itself.

Label: standard method (orchestration). Nothing here is physics.

The order it runs.

1. Parse the command and refuse impossible combinations (no input; two sequencing inputs; a BAM without its reference FASTA; a site

table without its pipeline; both kinds of reference list).

2. **Specimen check** (`stage_0_intake.specimen_refusal`). The chain reads whole blood, buffy coat (read on the whole-blood path,

labeled preliminary) and isolated neutrophils. Any other specimen is refused before a byte is read.

3. **Stage 0, intake**, for an IDAT pair: 0.1 arrival, 0.2 manifest, 0.3 integrity hash. A quarantine at any of these stops the run

(exit code 2) before calibration. An EPIC v2 header is refused here: the chain's references are EPIC v1, and Stage 1's calibrator does not read EPIC v2.

4. **The QC hand-off** (`stage_0_1_qc_handoff.decode_qc_inputs`) reads the array's control probes, probe intensities, negative-control

background, bead counts and sex probes straight from the IDAT files. If the array manifest cannot be loaded, the machine is at fault, not the array: exit code 3, and the array is not judged. If the decoder fails on the files, the array is quarantined as a corrupt IDAT.

5. **Stage 0, the QC gates**: 0.4 control probes, 0.5 detection p-values, 0.6 bead counts, 0.7 call rate, 0.8 sex, 0.7b platform

coverage, then 0.9 the decision gate. A quarantine prints every reason in five parts (measured, limit, where the limit comes from, what it means, what to do) and stops the run.

6. **Stage 1, calibration** (`stage_1_idat_calibration.calibrate_idat_to_beta`): the IDAT pair becomes one beta value per site, with

the probes at background removed. Stage 1's own quality numbers are recorded beside Stage 0's, never over them, and never gate.

7. **Stage 0.7b again**, now with the calibrated values: `run_sample.py` counts the share of Met-A's 6,000 neutrophil identity sites

the array measured and hands it to 0.7b. On a 450K array 0.7b quarantines below 80 %; on EPIC, the only platform the chain reads, it records `NA_EPIC` and passes. Met-A itself withholds its reading when fewer than 90 % of its sites were measured (Stage M).

8. **The conductor** (`conductor_v3.run_neutrophil`): the platform check, Stage A (the cell mix), Stage M (Met-A), Stage MC (the

C-Score) and Stage T (the same-run tare).

9. **Chain v4 additions** when the array was read: Stage D21 (`stage_v4.stage_d21`, the IAM-Deconvolver's cell shares against

IAM-Atlas v2.1) and the three skies (the C-Score's departures on the sphere, the healthy neutrophil of IAM-Atlas v2.1 at the same sites, and the overlay).

10. **IAM-A** when sequencing data is given: Stage Q0 intake on a `.pat` file, then Stage Q2 (`stage_v4.count_sequencing` and

`stage_v4.stage_q2`), the held error and the loose share counted in one pass and read against the sample's own reference. The retired position reading (Stage Q) runs only with `--site-table` or `--dev-iama-position`, labeled development.

11. **Stage 12b**, the difference between two draws of the same person, when `--prior-betas` is given; refused unless both draws carry

the same identifier hash, array type and pipeline.

12. **Development readings**, only when a `--dev-*` flag asks for them, written under `bundle["development"]` and never part of the

reading.

13. **The report** (`report_v4.build`): one HTML page with a researcher view and a plain view, and two PDFs. Then the identifier is

replaced by its hash everywhere in the bundle and the ledger, the bundle is written as JSON, and one row is appended to the evidence ledger.

Exit codes. 0 a report was written (it may still withhold a reading, and says why); 2 the sample was quarantined at intake and nothing was scored; 3 the machine is missing a file it needs (the array manifest) and the array was not judged.

What it reads. The command line; the IDAT pair or beta table or sequencing file; `--ref-table` (the same-run healthy references: column A for Met-A, column C for the C-Score); `Runtime Matrices/Met_A_Floors/metA_floors_v1_3.json` for the 0.7b coverage count. Everything else is read by the stages.

How to break it. Give two sequencing inputs, a BAM without `--ref-fasta`, a specimen outside the list, an EPIC v2 array, a truncated IDAT: each must stop with a named reason, never a traceback. Release checks V1 to V10 run the one command end to end.

run_sample.py::_bio_root

Finds the `Biological_Physics` folder by walking up from this file until it finds a folder holding `MethylPhys`, rather than by counting levels. Written after a 2026-09-22 move changed every script's depth by one and a counted path silently stopped finding the record folder. Decides nothing about a sample.

Code: `MethylPhys_Interface/run_sample.py` line 39 (16 lines) · arguments: start · calls: array run 1, bam run 1

run_sample.py::_assign_run_id

Gives the run its identifier, `RUN-YYYYMMDD-NN`: the UTC date and the next number for that day, counted from the ledger the run is about to append to. A run makes no claim and passes no bar; it is one execution on one specimen, and the identifier lets a reading be found again.

Code: `MethylPhys_Interface/run_sample.py` line 59 (19 lines) · arguments: ledger_path · calls: array run 1, bam run 1

run_sample.py::_covariates

Collects the phenotype and covariates for the run: a JSON object from `--covariates`, then each `--covariate KEY=VALUE` on top. They are kept in the bundle and the custody record and are never printed in the report's prose. No stage reads them.

Code: `MethylPhys_Interface/run_sample.py` line 80 (16 lines) · arguments: a · calls: array run 1, bam run 1

run_sample.py::_sha12

The first 12 hexadecimal characters of a file's SHA-256, used to fingerprint each frozen input in `_versions`. Returns nothing if the file cannot be read.

Code: `MethylPhys_Interface/run_sample.py` line 98 (10 lines) · arguments: path · calls: array run 18, bam run 18

run_sample.py::_versions

Records everything that can change a reading: the UTC time, the Python and methylprep versions, the chain's git commit and whether the working copy had uncommitted changes, and a short hash and size of each frozen matrix and stage file. Two runs months apart can be pooled only if these match.

Code: `MethylPhys_Interface/run_sample.py` line 110 (55 lines) · arguments: chain_dir · calls: array run 1, bam run 1 · files named: `Runtime Matrices/IAM_A_Positions/hg19_cpg_chrom_ranges.json`, `Runtime Matrices/IAM_A_Positions/iama_positions_v2.json`, `Runtime Matrices/Intake/intake_thresholds_v1.json`, `Runtime Matrices/Met_A_Floors/blood_composition_EPIC_v1.json`, `Runtime Matrices/Met_A_Floors/metA_floors_v1_3.json`, `Runtime Matrices/Met_A_Floors/metA_floors_v1_3_loo.csv`, `Runtime Matrices/Met_A_Floors/neutrophil_reference_v1_2.json`, `Runtime Matrices/Met_A_Floors/noise_gate_EPIC_v1.json`, `Runtime Matrices/Met_A_Floors/noise_sites_EPIC_v1.json`, `compact_manifest.json`, `in`

compact_manifest.json, transfer_v21_final.json

run_sample.py::_why

Prints and keeps one stop's explanation in five parts (what was measured, the limit, where the limit comes from and whether it is provisional, what it means, what to do), for every stop outside Stage 0's own gates: a refused specimen, the early intake stops, EPIC v2, a missing manifest (exit code 3), a corrupt IDAT, Stage 1's platform refusal and the quarantine after calibration. The report shows the same rows under "Why the chain stopped or flagged this sample". Release check V11 fails if any exit with code 2 or 3 has no explanation in its own block.

Code: MethylPhys_Interface/run_sample.py line 167 (8 lines) · arguments: rec, check, outcome, measured, limit, source, means, todo, provisional · calls: not called on the traced runs

run_sample.py::_hash_id

Replaces a typed identifier with the first 32 characters of its SHA-256. An identifier that already looks like a hash (16 or more hexadecimal characters) is kept. The engine never stores a cleartext identifier.

Code: MethylPhys_Interface/run_sample.py line 177 (5 lines) · arguments: x · calls: array run 1, bam run 1

run_sample.py::_redact

Walks the whole bundle (every string, dictionary key and list) and replaces each whole-token occurrence of the typed identifier with its hash, so neither the bundle nor the ledger carries the typed name. The printed report keeps the typed name for the operator.

Code: MethylPhys_Interface/run_sample.py line 184 (11 lines) · arguments: obj, typed, hid · calls: array run 4, bam run 4

run_sample.py::_redact.f

The recursive step of _redact: a string gets the replacement, a dictionary and a list are walked, anything else is returned as is.

Code: MethylPhys_Interface/run_sample.py line 189 (5 lines) · arguments: v · calls: array run 865, bam run 359

run_sample.py::_dev_run

Runs the development readings named by --dev-* flags (direction, trace cell, foreign cell, brightness interval, NILC-e, atlas e, B-cell reading, sky with the block-shuffle null, EPIC v2 through SeSAMe). Each writes its own record under bundle["development"], labeled DEVELOPMENT, and an error in one is caught and recorded. None of them changes the reading, the gauge or the tare.

Code: MethylPhys_Interface/run_sample.py line 197 (40 lines) · arguments: a, o, beta, intake · calls: array run 1, bam run 1

run_sample.py::main

The orchestration described above, from parsing the command to writing the ledger row. Decides which stages run, stops the run with exit code 2 or 3 at the stops listed, carries the same-run references (columns A and C) into the conductor, adds the v4 stages, and hands everything to the report.

Code: MethylPhys_Interface/run_sample.py line 239 (461 lines) · arguments: none · calls: array run 1, bam run 1 · constants read: BIO, ENG · files named: MethylPhys/chain/Runtime Matrices/Met_A_Floors/metA_floors_v1_3.json, _bundle.json, a wgbstools .pat / .pat.gz, pip install -r Biological_Physics/MethylPhys/chain/requirements.txt

run_sample.py::main._stopped

True when an intake record's status begins with QUARANTINE. Intake checks it after every step, so a quarantine ends intake at its cause. Without it a later step overwrote the status, and the verdict named a downstream symptom (seen 2026-09-23: an array-type mismatch was reported as a detection failure).

Code: `MethylPhys_Interface/run_sample.py` line 341 (5 lines) · arguments: r · calls: array run 3

`run_sample.py::main._ctx`

Builds the comparison record Stage 12b needs from a draw: the identifier hash, the array type and the calibration pipeline.

Code: `MethylPhys_Interface/run_sample.py` line 646 (2 lines) · arguments: b, it, pipe · calls: not called on the traced runs

2. Stage 0, intake: `stage_0_intake.py`

What it is for. Deciding whether this pair of files is a usable array from the specimen it claims to be, before anything is calibrated or read. A quarantine here means nothing about the biology was looked at.

Label: standard method. These are the array maker's and the field's quality checks (control probes, detection against background, bead counts, call rate, a sex check from the X and Y signals), applied to one array on its own. None of the limits comes from IAM's physics. Every limit lives in a named constant or in `Runtime Matrices/Intake/intake_thresholds_v1.json`, never as a bare number in a gate.

The specimen check (before Stage 0). `specimen_refusal` accepts whole blood (and the words blood, peripheral blood, venous blood), isolated, sorted or purified neutrophils, buffy coat (read on the whole-blood path, labeled as its own specimen, preliminary), and the constructed DNA mixtures used as test material. PBMC, other sorted cell fractions, bone marrow, cell lines, an unstated specimen and anything else are refused with the reason, and nothing is read.

The steps, in the order the run calls them.

Step	What it measures	Outcome	Limit and source
0.1 arrival	the manifest's required fields (Sentrix ID, array type, identifier, intake date, specimen); both channels present; each IDAT at least 1 MB; the array type in the IDAT header against the declared one	quarantine on a missing field, a missing channel, a truncated file, or a 450K/EPIC family swap; a note on an EPIC v1/v2 subtype difference; a soft warning on the same Sentrix ID taken in within 24 h	fixed rules (SOP 11): `MIN_PLAUSIBLE_IDAT_BYTES` 1,000,000; header address count below 800,000 is 450K, below 1,080,000 EPIC v1, above EPIC v2
0.2 manifest	builds the immutable per-sample record; the identifier must already be a hash	quarantine on a cleartext identifier (spaces, an @, under 16 characters) or a missing core field	fixed rules (SOP 12)
0.3 integrity	SHA-256 of both files against earlier intakes of the same Sentrix ID in this intake log	identical bytes again: re-transmission, the run stops; same Sentrix ID with different bytes: a re-run of a fresh array, read	fixed rule (SOP 13)
0.4 control probes	bisulfite conversion (C/(C+U) per matched BS Conversion I pair, median over pairs), hybridization (high against low concentration, green) and extension (green C and G against red A and T) controls, from the array's own control addresses	a failed hybridization or extension stops the array; the bisulfite number is printed and never stops an array on its own	`HYB_HIGH_LOW_MIN_RATIO` 2.0 and `EXTENSION_BALANCE_RANGE` 0.2 to 5.0, both **provisional**; bisulfite at least 0.95, **provisional** (as written it refuses 731 of 731 healthy arrays)
0.5 detection	per probe, $p = 1 - \Phi((I - \mu_bg)/\sigma_bg)$ against this array's negative-control background; the share of probes with $p \le 0.01$	above 0.99 passes; 0.93 to 0.99 is read with a penalty flag; below 0.93 stops	`intake_thresholds_v1.json` detection
0.6 bead counts	the share of probes resting on at least 3 beads	below 0.995 a penalty flag; never stops	`BEAD_COUNT_MIN` 3; `intake_thresholds_v1.json` bead

Step	What it measures	Outcome	Limit and source
0.7 call rate	the share of probes passing both detection and bead count	0.98 or more passes; 0.93 to 0.98 penalty; below 0.93 stops	`intake_thresholds_v1.json` call_rate; the 0.93 line was set by the author on 2026-09-27 from 48 arrays, 12 from each of four commissioning laboratories
0.7b coverage	the platform tag; on a 450K array, the share of the reference sites measured	on 450K, below 0.80 stops; on EPIC, `NA_EPIC` (passes)	`HM450_COVERAGE_MIN` 0.80
0.8 sex	median log2 intensity of the Y probes minus the X probes	Y − X below −2 is female, otherwise male; a mismatch with a declared sex stops the array; with no declared sex the array's sex is recorded	`SEX_GETSEX_CUTOFF` −2.0, the rule of the minfi package's getSex (SOP 18)
0.9 decision gate	every step's outcome	any hard failure: QUARANTINE; any penalty: PROCEED_WITH_PENALTY; else PROCEED	a detection or call rate that could not be measured is a hard failure, never a pass (added 2026-09-27 after a low-signal laboratory had walked through for a week)

Every stop explains itself. `explain_stage0` attaches, for every check that stopped, penalized or deferred the array: what was measured on this array, the limit, where the limit comes from and whether it is provisional, what it means and what to do. `run_sample.py` prints these on a quarantine. Release check V9 builds each kind of stop and fails if any one is not explained.

Built from what, tested on what. The detection and call-rate lines were measured on 48 arrays, 12 from each of the four laboratories of Met-A's commissioning panel (`intake_thresholds_v1.json`, `_meta`): Uppsala 0.985, Karolinska 0.975, UCLA 0.953, Munich 0.878. The detected fraction, not the signal-to-background ratio, separated the low-signal laboratory. The control-probe limits are provisional and have not been set from healthy arrays; setting them is a to-do step. Until then a provisional limit is printed with the word PROVISIONAL wherever it is used.

How to break it. Hand it a single channel, a 900 KB IDAT, a 450K pair declared as EPIC, the same pair twice under one Sentrix ID, a declared sex that disagrees with the array, or a low-signal array: each must quarantine with its reason printed in five parts. The file's own self-test (`python3 stage_0_intake.py`) runs every gate on constructed inputs.

`stage_0_intake.py::_intake_thresholds`

Loads `intake_thresholds_v1.json` from the chain's `Runtime Matrices/Intake` folder (never from a RETIRED folder) when the module is imported. Stage 0 refuses to start without it, so no gate can run on a number typed into the code.

Code: `stage_0_intake.py` line 35 (8 lines) · arguments: none · calls: array run 1, bam run 1 · files named: `intake_thresholds_v1.json`

`stage_0_intake.py::normalise_specimen`

Lowercases the specimen text, turns underscores into spaces and collapses repeated spaces, so `Whole_Blood` and `whole blood` are the same specimen.

Code: `stage_0_intake.py` line 77 (2 lines) · arguments: specimen · calls: array run 5

`stage_0_intake.py::specimen_refusal`

Returns nothing when the specimen is accepted, otherwise the plain refusal: which specimen, why the chain has no reference for it, and what is accepted. Called by `run_sample.py` before any file is read.

Code: `stage_0_intake.py` line 81 (8 lines) · arguments: specimen · calls: array run 2 · constants read: `ACCEPTED_SPECIMENS`, `REFUSED_SPECIMEN_REASONS`

`stage_0_intake.py::_infer_platform`

Turns the IDAT header's address count into a platform: below 800,000 is 450K, below 1,080,000 EPIC v1, otherwise EPIC v2.

Code: `stage_0_intake.py` line 101 (8 lines) · arguments: nsnps · calls: array run 2

`stage_0_intake.py::read_idat_nsnps`

Reads only the start of the green IDAT: the four-byte `IDAT` mark, the field table, and field 1000 (the number of addresses). It never decodes intensities. Returns the count and a status (OK, NOT_IDAT, NO_NSNPS_FIELD, FILE_NOT_FOUND or a read error).

Code: `stage_0_intake.py` line 113 (24 lines) · arguments: path · calls: array run 2

`stage_0_intake.py::step_0_1_idat_arrival`

Step 0.1: checks the manifest's required fields and array type, that both channels exist, that each file is at least 1 MB, and that the header's platform family agrees with the declared one; warns on a repeat intake within 24 hours. Stamps a run identifier and the arrival time, and logs the result.

Code: `stage_0_intake.py` line 141 (89 lines) · arguments: manifest_entry, grn_path, red_path, intake_log_path · calls: array run 1 · constants read: `MIN_PLAUSIBLE_IDAT_BYTES`, `REQUIRED_MANIFEST_FIELDS`, `VALID_ARRAY_TYPES`

`stage_0_intake.py::_log_intake`

Appends one row per intake step to the intake log, when one is given: time, run identifier, Sentrix ID, declared and detected array type, specimen, declared sex and age, status and flags.

Code: `stage_0_intake.py` line 234 (10 lines) · arguments: intake_log_path, result · calls: array run 1

`stage_0_intake.py::_is_duplicate_within_24h`

True when the intake log already holds the same Sentrix ID from the last 24 hours. Only a warning; the array still advances.

Code: `stage_0_intake.py` line 246 (21 lines) · arguments: intake_log_path, sentrix_id · calls: array run 1

`stage_0_intake.py::_is_null_answer`

For storing questionnaire answers: a blank or "prefer not to say" becomes no answer; "No" is kept as a real answer.

Code: `stage_0_intake.py` line 329 (3 lines) · arguments: v · calls: not called on the traced runs · constants read: `_NULL_FOR_STORAGE`

`stage_0_intake.py::_answer_present`

For the questionnaire's context modes: true only for an affirmative answer (not blank, not "prefer not to say", not No, none or not applicable).

Code: `stage_0_intake.py` line 334 (5 lines) · arguments: v · calls: not called on the traced runs · constants read: `_NEGATIVE`, `_NULL_FOR_STORAGE`

`stage_0_intake.py::_age_from_dob`

Age in decimal years from a date of birth and the intake date, rounded to 0.1.

Code: `stage_0_intake.py` line 341 (8 lines) · arguments: dob, on_date · calls: not called on the traced runs

`stage_0_intake.py::parse_questionnaire`

Maps a digital questionnaire's answers to the stored covariate record: each field, the age from the date of birth, a smoking bin, and a list of context modes. The record is stored with the manifest. No stage of the current chain reads

these covariates or modes; the stage names in its field table belong to an earlier engine.

Code: `stage_0_intake.py` line 351 (18 lines) · arguments: answers, intake_date · calls: not called on the traced runs · constants read: `COVARIATE_FIELDS`, `_CONTEXT_MODE_TRIGGERS`

`stage_0_intake.py::_looks_like_cleartext_pii`

True when an identifier looks like a name rather than a hash: empty, containing a space or an @, shorter than 16 characters once dashes and underscores are removed, or not purely letters and digits.

Code: `stage_0_intake.py` line 371 (9 lines) · arguments: patient_id · calls: array run 1

`stage_0_intake.py::_write_manifest`

Writes the per-sample manifest as `patient_manifest_<run id>.json` in the manifest folder, when one is given.

Code: `stage_0_intake.py` line 382 (8 lines) · arguments: manifest_dir, record · calls: array run 1 · files named: `Write the immutable per-sample patient_manifest.json`

`stage_0_intake.py::step_0_2_manifest_creation`

Step 0.2: builds the immutable manifest from the 0.1 record (and questionnaire answers, when given), quarantines a cleartext identifier or a missing core field, and writes the file.

Code: `stage_0_intake.py` line 392 (40 lines) · arguments: step_0_1_result, questionnaire_answers, manifest_dir, intake_date · calls: array run 1

`stage_0_intake.py::_sha256_file`

The SHA-256 of a whole file, read in 1 MB pieces.

Code: `stage_0_intake.py` line 440 (6 lines) · arguments: path · calls: array run 2

`stage_0_intake.py::_prior_hashes_for_sentrix`

The pairs of file hashes earlier intakes recorded for the same Sentrix ID in the integrity log.

Code: `stage_0_intake.py` line 448 (14 lines) · arguments: integrity_log_path, sentrix_id · calls: array run 1

`stage_0_intake.py::_log_integrity`

Appends one row to the integrity log (also used for the decision gate's verdict row).

Code: `stage_0_intake.py` line 464 (6 lines) · arguments: integrity_log_path, row · calls: array run 1

`stage_0_intake.py::step_0_3_integrity_hash`

Step 0.3: hashes both files and compares them with earlier intakes of the same Sentrix ID. Identical bytes again is a re-transmission and stops the run; the same Sentrix ID with different bytes is flagged as a fresh re-run and advances.

Code: `stage_0_intake.py` line 472 (37 lines) · arguments: record, grn_path, red_path, integrity_log_path · calls: array run 1

`stage_0_intake.py::extract_control_probes`

Gets the control-probe medians by class from the IDAT pair through the QC hand-off (`stage_0_1_qc_handoff.decode_qc_inputs`), the same decoder Stage 1 uses. The array's own controls only.

Code: `stage_0_intake.py` line 530 (7 lines) · arguments: grn_path, red_path, array_type · calls: not called on the traced runs

`stage_0_intake.py::validate_control_probes`

Step 0.4's arithmetic. Bisulfite: `bs1 / (bs1 + bs2)` against 0.95, where the QC hand-off passes in the matched-pair conversion efficiency and its complement under the keys `bisulfite_conversion_I_median` and `bisulfite_conversion_II_median` (so the ratio is the efficiency itself; the key names are older than the pair method). Hybridization: the high-concentration median over the low-concentration median, at least 2. Extension: the green-channel (C and G) extension median over the red-channel (A and T) median, between 0.2 and 5; the record calls these "meth" and "unmeth", which they are not. When the bisulfite number is the only failure, the verdict is PROVISIONAL and the array advances, because that limit has not been set from healthy arrays. Records the raw medians the gates tested.

Code: `stage_0_intake.py` line 539 (41 lines) · arguments: control_summary · calls: array run 2 · constants read: `BS_CONVERSION_MIN`, `BS_THRESHOLD_CALIBRATED`, `EXTENSION_BALANCE_RANGE`, `HYB_HIGH_LOW_MIN_RATIO`

stage_0_intake.py::step_0_4_control_probe_validation

Step 0.4: runs `validate_control_probes` on the hand-off's control summary and stamps the verdict and its numbers. With no decoder available it marks the check deferred rather than inventing a pass.

Code: `stage_0_intake.py` line 582 (20 lines) · arguments: record, grn_path, red_path, control_summary · calls: array run 1

stage_0_intake.py::_normal_sf

The normal survival function, 1 – Φ(z), from scipy when it is installed, otherwise from the complementary error function.

Code: `stage_0_intake.py` line 621 (9 lines) · arguments: z · calls: array run 2

stage_0_intake.py::compute_detection_p

Per probe, the detection p-value 1 – Φ((I – μ_bg)/σ_bg), with μ_bg and σ_bg this array's negative-control background.

Code: `stage_0_intake.py` line 632 (6 lines) · arguments: probe_intensities, mu_bg, sigma_bg · calls: array run 2

stage_0_intake.py::validate_detection_p

The share of probes with detection p ≤ 0.01, graded against the thresholds file: above 0.99 PASS, from 0.93 BORDERLINE, below FAIL.

Code: `stage_0_intake.py` line 640 (15 lines) · arguments: detection_p_array · calls: array run 2 · constants read: `DETECTION_BORDERLINE_FRACTION`, `DETECTION_PASS_FRACTION`, `DETECTION_P_THRESHOLD` · files named: `intake_thresholds_v1.json`

stage_0_intake.py::step_0_5_detection_pvalue_qc

Step 0.5: computes and grades detection. Without intensities the check is deferred, and a deferred detection never advances.

Code: `stage_0_intake.py` line 657 (22 lines) · arguments: record, probe_intensities, neg_control_stats · calls: array run 1

stage_0_intake.py::validate_bead_count

The share of probes with at least 3 beads: 0.995 or more PASS, otherwise a warning. Never stops an array.

Code: `stage_0_intake.py` line 689 (8 lines) · arguments: bead_counts, min_beads · calls: array run 1 · constants read: `BEAD_COUNT_MIN`, `BEAD_PASS_FRACTION`

stage_0_intake.py::step_0_6_bead_count_qc

Step 0.6: grades the bead counts and flags a low count. Always advances.

Code: `stage_0_intake.py` line 699 (14 lines) · arguments: record, bead_counts · calls: array run 1

`stage_0_intake.py::validate_call_rate`

The share of probes passing both detection and bead count, graded: 0.98 or more PASS, from 0.93 BORDERLINE, below FAIL.

Code: `stage_0_intake.py` line 723 (9 lines) · arguments: n_passing_both, n_total · calls: array run 2 · constants read: `CALL_RATE_BORDERLINE`, `CALL_RATE_PASS`

`stage_0_intake.py::step_0_7_call_rate`

Step 0.7: combines the detection and bead masks probe by probe and grades the call rate. Without masks the check is deferred, and a deferred call rate never advances.

Code: `stage_0_intake.py` line 734 (22 lines) · arguments: record, detection_pass_mask, bead_pass_mask · calls: array run 1

`stage_0_intake.py::step_0_7b_platform_coverage`

Step 0.7b: stamps the platform tag. On a 450K array it stops below 80 % of the reference sites; on EPIC it records `NA_EPIC` and passes.

Code: `stage_0_intake.py` line 765 (25 lines) · arguments: record, reference_cpg_coverage · calls: array run 2 · constants read: `HM450_COVERAGE_MIN`

`stage_0_intake.py::predict_sex`

Female when the median log2 Y intensity minus the median log2 X intensity is below −2, otherwise male (minfi's getSex rule).

Code: `stage_0_intake.py` line 799 (2 lines) · arguments: log2_x_median, log2_y_median, cutoff · calls: array run 1 · constants read: `SEX_GETSEX_CUTOFF`

`stage_0_intake.py::validate_sex`

Compares the predicted sex with a declared one. No declared sex: the array's sex is recorded and nothing is compared. A mismatch stops the array.

Code: `stage_0_intake.py` line 803 (7 lines) · arguments: log2_x_median, log2_y_median, declared_sex · calls: array run 1

`stage_0_intake.py::step_0_8_sex_check`

Step 0.8: runs the sex check on the hand-off's X and Y medians and stamps the result.

Code: `stage_0_intake.py` line 812 (15 lines) · arguments: record, sex_intensities · calls: array run 1

`stage_0_intake.py::_fmt`

Formats a measured number for an explanation, or says "not measured".

Code: `stage_0_intake.py` line 841 (2 lines) · arguments: x, nd · calls: array run 2

`stage_0_intake.py::explain_stage0`

Builds the five-part explanation of every check that stopped, penalized or deferred the array (measured on this array, the limit, the limit's source and whether it is provisional, what it means, what to do). Changes no decision.

Code: `stage_0_intake.py` line 843 (54 lines) · arguments: record, hard_fail, borderline, deferred · calls: array run 2 · constants read: `BEAD_COUNT_MIN`, `BEAD_PASS_FRACTION`, `BS_CONVERSION_MIN`, `BS_THRESHOLD_CALIBRATED`, `CALL_RATE_BORDERLINE`, `CALL_RATE_PASS`, `DETECTION_BORDERLINE_FRACTION`, `DETECTION_PASS_FRACTION`, `EXTENSION_BALANCE_RANGE`, `HM450_COVERAGE_MIN`, `HYB_HIGH_LOW_MIN_RATIO`, `SEX_GETSEX_CUTOFF`, `_TH_SRC`

`stage_0_intake.py::explain_stage0.add`

Adds one explanation to the list.

Code: `stage_0_intake.py` line 845 (2 lines) · arguments: check, outcome, measured, limit, source, provisional, means, todo · calls: array run 2

`stage_0_intake.py::step_0_9_decision_gate`

Step 0.9: sorts every step's outcome into hard failures, penalties and deferrals and gives the verdict (QUARANTINE, PROCEED_WITH_PENALTY or PROCEED). A deferred detection or call rate is a hard failure; a deferred bead count or provisional bisulfite number is recorded. Attaches the explanations and logs the verdict.

Code: `stage_0_intake.py` line 899 (80 lines) · arguments: record, verdict_log_path · calls: array run 2

3. The QC hand-off: `stage_0_1_qc_handoff.py`

What it is for. Stage 0's gates 0.4 to 0.8 need intensities, and intensities come from decoding the IDAT files. This module decodes the pair just far enough to give those gates real numbers from the array itself, using the same decoder Stage 1 uses (methylprep) and the control addresses of the array's own manifest. Nothing from any other sample enters.

Label: standard method (Illumina's control probes, read by design type the way the minfi package reads them).

What it returns.

- **Control summary.** Bisulfite conversion: for each matched BS Conversion I pair (C1 with U1, C2 with U2, ...), the converted signal

over converted plus unconverted, C/(C+U), and the median over the pairs. Hybridization: the medians of the high- and low-concentration controls (green). Extension: the median of the C and G controls (green) and of the A and T controls (red).

- **Negative-control background.** The negative controls summed over both channels, summarized robustly as the median and 1.4826 times

the median absolute deviation (MAD). The mean and standard deviation are returned beside them. The robust form was adopted on 2026-09-23 after a few EPIC negative controls carrying real signal (one at 29,110 against a median of 134) pushed the standard deviation to 1,676 and failed half the probes of a good array; on a clean 450K array the two estimates agree.

- **Per-probe total intensity.** A type II probe is read at its one address in both channels; a type I probe at its two addresses in

its own color only (the other color would add only background).

- **Bead counts**, per probe the smaller of the two channels' counts (and of the two addresses for type I).
- **Sex intensities**: log2 of the median total intensity of the X probes and of the Y probes.

How to break it. A truncated or corrupt IDAT makes the decoder fail; `run_sample.py` then quarantines the array as a corrupt IDAT. A missing manifest on the machine is an environment failure (exit code 3), never a judgment of the array.

`stage_0_1_qc_handoff.py::_manifest`

Loads the array type's manifest from methylprep once and keeps it for the rest of the run.

Code: `stage_0_1_qc_handoff.py` line 26 (9 lines) · arguments: array_type · calls: array run 2 · constants read: `_MANIFEST`

`stage_0_1_qc_handoff.py::_channel_means`

Reads one IDAT file (gzipped or plain) with methylprep and returns the mean intensity at every address and, when asked, the bead count.

Code: `stage_0_1_qc_handoff.py` line 37 (11 lines) · arguments: path, channel, want_beads · calls: array run 2

`stage_0_1_qc_handoff.py::_median`

The median intensity over a list of control addresses, ignoring addresses the file lacks; nothing if none are present.

Code: `stage_0_1_qc_handoff.py` line 50 (3 lines) · arguments: series, addresses · calls: array run 4

`stage_0_1_qc_handoff.py::decode_qc_inputs`

Builds everything Stage 0's gates read, as listed above, from the two files and the manifest. Returns the probe identifiers in the same order as the intensities and bead counts, so Stage 1's call rate can line them up later.

Code: `stage_0_1_qc_handoff.py` line 55 (125 lines) · arguments: grn_path, red_path, array_type · calls: array run 1

`stage_0_1_qc_handoff.py::decode_qc_inputs.ctl_addr`

The manifest addresses of one control type, optionally narrowed to extended types containing (or not containing) a word.

Code: `stage_0_1_qc_handoff.py` line 62 (9 lines) · arguments: control_type, contains, excludes · calls: array run 5

`stage_0_1_qc_handoff.py::decode_qc_inputs._at`

The intensity at each probe's address, with a gap where the probe has no such address.

Code: `stage_0_1_qc_handoff.py` line 135 (6 lines) · arguments: series, addr · calls: array run 8

`stage_0_1_qc_handoff.py::decode_qc_inputs.sex_median`

log2 of the median total intensity over the X (or Y) probes the array measured, and how many probes that was.

Code: `stage_0_1_qc_handoff.py` line 166 (5 lines) · arguments: which · calls: array run 2

4. Stage 1, calibration: `stage_1_idat_calibration.py`

What it is for. Turning one array's raw intensities into one calibrated methylation value (beta) per site, from this array alone, and removing every probe that reads at background. Without calibration the methylated peak sits near 0.85 instead of 0.95 and every later number moves.

Label: standard method. Noob (normal-exponential out-of-band background correction with dye-bias correction), as implemented in the methylprep package, run on one array at a time with its own control and out-of-band probes. No cohort and no reference enters.

The formula. For each probe, beta = M / (M + U + offset) after noob correction of the methylated (M) and unmethylated (U) intensities. Detection: methylprep's poobah p-value, each probe against this array's own out-of-band background; a probe with p above 0.05 is removed before any stage reads a value ("no measurement at background").

What it pins. The commissioned numbers were made with methylprep 1.7.1 and three Illumina manifests, fixed by their SHA-256 (`METHYLPREP_VERSION`, `MANIFEST_SHA256`). Stage 1 refuses to run on another version or a changed manifest, before and after methylprep fetches its files. Development work can override this with `IAM_ALLOW_UNPINNED_STAGE1=1`; the results are then not the commissioned chain's.

What else it returns. The detection summary (probes measured, detected, removed); the control medians; and the array's own noise, from its SNP (rs) probes. Those probes cluster at 0, 0.5 and 1 by genotype, so their spread around each cluster is pure measurement noise: $\sigma^2(\beta) = a + b \cdot \beta(1 - \beta)$, with a the mean variance of the two homozygous clusters and $b = 4 \cdot$(variance of the heterozygous cluster $-$ a). The sky uses this as the array's own noise term.

How to break it. Run with methylprep 1.7.2, or edit one byte of a cached manifest: Stage 1 must refuse. An EPIC v2 pair reaching Stage 1 is refused as an array type it does not calibrate; a truncated file is quarantined as a corrupt IDAT.

`stage_1_idat_calibration.py::_check_pins`

Refuses to run unless methylprep is version 1.7.1 and every cached manifest matches its pinned SHA-256 (or the development override is set). Called before calibration and again after methylprep fetches any missing manifest.

Code: `stage_1_idat_calibration.py` line 38 (11 lines) · arguments: none · calls: array run 2 · constants read: `MANIFEST_SHA256`, `METHYLPREP_VERSION`

`stage_1_idat_calibration.py::_ensure_methylprep`

Imports methylprep, installing the pinned version into the running Python once if it is missing, and explains what to do when the install or import fails.

Code: `stage_1_idat_calibration.py` line 49 (43 lines) · arguments: none · calls: array run 1 · constants read: `METHYLPREP_VERSION` · files named: `pip install -r Biological_Physics/MethylPhys/chain/requirements.txt`

`stage_1_idat_calibration.py::_detect_array_type`

Reads the green IDAT with methylprep and maps its address count to methylprep's array type; also returns the barcode.

Code: `stage_1_idat_calibration.py` line 94 (10 lines) · arguments: grn_path · calls: array run 1

`stage_1_idat_calibration.py::calibrate_idat_to_beta`

Copies the pair into a private temporary folder with a one-row sample sheet (so methylprep processes this array alone), runs noob with poobah detection and the control and SNP probes exported, keeps the cg probes with poobah $p \leq 0.05$, and returns the betas with the record described above. The temporary folder is always deleted.

Code: `stage_1_idat_calibration.py` line 106 (106 lines) · arguments: grn_path, red_path, array_type, verbose, mask_detection, return_mask, return_snp · calls: array run 1 · files named: `_processed.csv`, `control_probes.pkl`, `samplesheet.csv`, `the tag beta_scale_maps_v1.json`

`stage_1_idat_calibration.py::calibrate_idat_to_beta.med`

The median of one control type in one channel, from methylprep's exported control table.

Code: `stage_1_idat_calibration.py` line 176 (2 lines) · arguments: t, col · calls: array run 7

`stage_1_idat_calibration.py::_stage_idat`

Copies an IDAT into the staging folder under the name methylprep expects, unzipping it if it is gzipped.

Code: `stage_1_idat_calibration.py` line 214 (8 lines) · arguments: src, dst · calls: array run 2

5. The conductor: `conductor_v3.py`, and Met-A's reading: `stage_m_met_a.py`

What it is for. The conductor takes one array's calibrated values and makes the array reading: the platform check, Stage T step 1 (self-tare II), Stage A (the cell mix), Stage M (Met-A), the noise index, Stage T step 2 (the same-run tare), and Stage MC (the C-Score). It returns one record per stage, each with its reason when it gives no number.

The order inside `run_neutrophil`.

1. Refuse a specimen outside the list (as `run_sample.py` already did) and an array that is not EPIC v1.
2. **Stage T step 1, self-tare II**: map this array's values onto the reference arrays' scale by its own fixed sites (below).
3. Isolated neutrophils: **Stage M** against the healthy reference. Whole blood (and buffy coat, and constructed mixtures): **Stage A**,

then **Stage M** against the expectation for this blood's own mix.

4. The **noise index** N, from the unmapped values.
5. **Stage T step 2**, the median tare against the same-run healthy references.
6. The **noise gate**: no gauge state when N cannot be formed, or when N is above the reference arrays' range and the reading is not tared.
7. The scope label: commissioned on isolated neutrophils; preliminary (labeled development) on whole blood and buffy coat.
8. **Stage MC**, the C-Score, and its own same-run tare; the commissioned label on whole blood.

The physics: what Met-A measures

Label: from IAM's physics. A cell holds a methylation pattern against thermal noise by paying for it. A site held fully methylated or fully unmethylated carries no uncertainty; a site that has drifted toward 50 % carries up to one bit. The entropy of one site's beta value is

$$H(\beta) = -\beta \log_2 \beta - (1 - \beta) \log_2 (1 - \beta),$$

and Met-A is the mean entropy at the cell type's identity sites divided by the healthy reference:

isolated neutrophils: $\text{Met-A} = \text{mean}_i\, H(\beta_i) / H_{ref}$, with $H_{ref} = 0.33026$ bits (6 healthy neutrophil arrays); whole blood: $\text{Met-A} = \text{mean}_i\, H(\beta_i) / \text{mean}_i\, H(e_i)$, with $e_i = \Sigma_g\, f_g\, \mu_{g,i}$.

Here f_g is this blood's share of cell group g (Stage A), and $\mu_{g,i}$ is the healthy mean of group g at site i, from purified cells. In whole blood the denominator is what a healthy blood with exactly this person's cell mix would read, so a change in the mix does not move Met-A; only a change in how well the cells hold their marks does.

The gauge. Healthy is 1 (the middle mark, measured). Normal is 0.95 to 1.05. The floor (derived: thermal kicks against about one ATP per site, M = 20.94) sits millions of times below any reading; the ceiling (derived: one bit per site, every identity site scrambled) sits about three times above healthy. What the physics fixes is the scale and its two ends. The healthy middle is measured once and never fitted to the sample.

The ceiling caveat (`_ceiling`). $H(\beta)$ peaks at $\beta = 0.5$. Met-A rises with loss of the pattern only while the methylated sites stay above 0.5. Every reading records the mean β of the cell's methylated sites and flags `past_entropy_ceiling` when it has fallen below 0.5, because past that point further loss lowers Met-A.

Stage A, the cell mix (whole blood)

Label: standard method (reference-based deconvolution by non-negative least squares, used in the field for over a decade).

The blood's shares of 8 cell groups (neutrophils, eosinophils, basophils, monocytes, B, NK, CD4 T, CD8 T) are solved from 963 marker sites: minimize |y – M f| with f ≥ 0, then f is scaled to sum to 1. M holds each group's mean β at each marker, from 91 purified-cell EPIC arrays of two published purified-blood-cell studies (GSE110554 and GSE167998), calibrated by this chain's own Stage 1.

- **Why its own reader, not the atlas:** the expectation e must come from the same platform and the same reference as the profiles it

multiplies, or any platform difference leaks into Met-A. IAM-Atlas v2.1 and the Deconvolver report the shares separately (Stage D21).

- **Why no marker is an identity site:** solving the mix from the sites Met-A reads would let a change in the cells' marks move the mix

that is supposed to explain them.

- **Gate:** at least 90 % of the 963 markers measured (867); otherwise no mix, no Met-A, and the reason gives the count.
- **Built from, tested on:** built from the 91 purified arrays (`chain_tests/blood_comp.py`). Rebuilt from GEO by the builder's own rule

on 2026-10-03, reproducing every group mean to 5 × 10■■, and again from a fresh clone (`metaa_rebuild`, PASS 2026-10-18).

Stage M, Met-A

- **Identity sites:** 6,000 EPIC sites the healthy neutrophil holds fixed, chosen on the purified neutrophil arrays (across-array SD

≤ 0.05; mean β 0.75 to 0.95, or 0.05 to 0.25; up to 3,000 per state). The isolated and whole-blood paths read the same 6,000.

- **The healthy reference (H_ref = 0.33026 bits):** the mean entropy at those sites on 6 purified healthy neutrophil arrays, each

physical array counted once. The file holding it is still named `metA_floors_v1_3.json`, and its key is `floor`; it is the middle mark, not the floor.

- **Held-out precision:** each of the 6 arrays read with the sites re-chosen on the other 5 (`metA_floors_v1_3_loo.csv`) gives 0.983 to

1.045. This is printed with every isolated reading.

- **Gate:** at least 90 % of the 6,000 sites measured; otherwise no Met-A, and the reason gives the count. In whole blood, a neutrophil

share of 0 also withholds the reading.

- **Detection at this person's mix:** the reading records how far a 1 % loss of the neutrophils' pattern, planted at this blood's own

neutrophil share, would move it (`shift_per_1pct_loss`). Stage T turns this into the person's detection limit.

- **The departure map** for the C-Score: per site, z_i = (H(β_i) – H(ref_i)) / s_i, with ref the healthy neutrophil (isolated) or the

expectation e (blood), and s_i the shrunk healthy SD of H among neutrophils.

The noise index and its gate

The array's own noise is read where no blood cell has anything to say: 48,528 EPIC sites that every purified blood group holds fixed (each group's mean ≤ 0.03 or ≥ 0.97 and SD ≤ 0.02; none an identity site). N is their mean entropy, read before self-tare II. **Gate:** with fewer than 90 % of these sites measured the noise is unknown and no gauge state

is shown, tared or not. With N above N_max = 0.149 (the top of the reference arrays' range, `noise_gate_EPIC_v1.json`), the state is shown only after the same-run tare.

Stage T, the tare

Label: standard method (normalization against references run the same way).

- **Step 1, self-tare II** (`dev_stages.selftare_map`, on every reading since 2026-10-04). Each probe design (type I, type II) is mapped

onto the 6 reference arrays' scale by this array's own anchors: β' = L_r + (β – L)(U_r – L_r)/(U – L), with L and U this array's mean β over its own fixed low and high sites of that design, and L_r and U_r the same means on the reference arrays (type I 0.0186 and 0.9820; type II 0.0555 and 0.9486). It maps only the 6,963 sites Stages A and M read (the 6,000 identity sites and the 963 markers). A design whose anchors are missing, or closer than 0.1, is left unmapped. No other array enters; nothing is fitted.

- **Step 2, the median tare.** A_rel = A / median(A of at least 3 healthy arrays run the same way: same slide, else same batch). Normal is

0.95 ≤ A_rel ≤ 1.05. The reference spread is the SD of the references' A over their median. The **detection limit** is 2 × spread / shift per 1 % loss: the smallest loss of the cell's pattern, in percent, this person's reading could show. A sample that is itself in the reference list is excluded from it. Fewer than 3 references: the reading is printed as untared, and the reason says what to give.

- **Why:** untared Met-A in whole blood carries a laboratory offset of several percent. The tare cancels it, which is why Met-A needs no

cohort: it compares the person with a prediction for their own blood and centers it on their own run.

Stage MC, the C-Score

Label: from IAM's physics, read with a standard statistic. Drift that comes from holding marks against noise is scattered: each site fails on its own. Loss that comes from a process acting on a region, such as a tumor's, is clustered along the genome.

- **The statistic:** take the departure map z in genomic order, average blocks of 10 consecutive sites, and take the variance of the

block means times the block length over the variance of z. For scattered departures it is 1. Divided by the healthy median of that number (1.0103, from the 6 reference arrays held out one at a time), healthy reads about 1.

- **Its tare:** the same rule as Met-A's: C_rel = C / median(C of at least 3 same-run healthy references).
- **The band (commissioned on whole blood, 2026-10-15):** 0.877 to 1.152, the 2.5 to 97.5 % range of tared C on 617 healthy arrays from

19 studies. One-sided: above 1.152 is a clustered departure; below 0.877 is less clustered than healthy, not a departure.

How to break it. Plant a 1 % loss of the neutrophils' pattern scattered across sites: Met-A must move and the C-Score must not (seen on Group-A's 278 arrays: Met-A by up to 0.036, the C-Score by at most 0.006). Plant the same loss in runs of neighboring sites: the C-Score must rise. Give 2 references: the tare must refuse. Remove 15 % of the identity sites: Met-A must withhold its reading and say how many were measured.

`conductor_v3.py::ref`

Loads the neutrophil reference (`neutrophil_reference_v1_2.json`) once: the 6,000 sites in genomic order, the healthy mean and shrunk SD of H at each, and the C-Score's block length and healthy baseline.

Code: `conductor_v3.py` line 62 (4 lines) · arguments: none · calls: array run 4 · constants read: `RM`, `_REF` · files named: `neutrophil_reference_v1_2.json`

conductor_v3.py::noise_sites

Loads the 48,528 noise sites once.

Code: conductor_v3.py line 70 (6 lines) · arguments: none · calls: array run 1 · constants read: RM, _NS · files named: Runtime Matrices/Met_A_Floors/noise_sites_EPIC_v1.json, noise_sites_EPIC_v1.json

conductor_v3.py::noise_gate

Loads N_max and its rule once.

Code: conductor_v3.py line 78 (5 lines) · arguments: none · calls: array run 1 · constants read: RM, _NG · files named: Runtime Matrices/Met_A_Floors/noise_gate_EPIC_v1.json, noise_gate_EPIC_v1.json

conductor_v3.py::noise_index

N = mean H(β) over the noise sites this array measured, or nothing when fewer than 90 % were measured; with the counts.

Code: conductor_v3.py line 84 (6 lines) · arguments: beta · calls: array run 1 · constants read: H, MIN_NOISE_FRACTION

conductor_v3.py::_bc

Loads the blood composition matrix (blood_composition_EPIC_v1.json) once: the 963 markers and 8 group means at each, and the 8 groups' healthy means at the 6,000 identity sites.

Code: conductor_v3.py line 92 (4 lines) · arguments: none · calls: array run 4 · constants read: RM, _BC · files named: blood_composition_EPIC_v1.json

conductor_v3.py::min_markers

The number of markers Stage A needs: 90 % of 963, rounded up (867).

Code: conductor_v3.py line 97 (2 lines) · arguments: none · calls: array run 2 · constants read: MIN_MARKER_FRACTION

conductor_v3.py::stage_a_composition

Stage A: solves the 8 shares by non-negative least squares on the measured markers and scales them to sum to 1; records the fit's mean absolute residual. Refuses, with the count, below 867 markers.

Code: conductor_v3.py line 100 (15 lines) · arguments: beta · calls: array run 1

conductor_v3.py::_clustering

The C-Score's statistic on a departure map: variance of block means of 10 sites (× √10) over the site variance; nothing with fewer than 10 blocks.

Code: conductor_v3.py line 116 (5 lines) · arguments: z, w · calls: array run 1

conductor_v3.py::_state

Normal from 0.95 to 1.05, otherwise above or below Normal.

Code: conductor_v3.py line 122 (2 lines) · arguments: A · calls: array run 1

conductor_v3.py::stage_m_blood

Stage M in whole blood: the expectation e for this blood's mix, Met-A = mean H(β) / mean H(e) over the measured identity sites, the shift a 1 % loss would make at this neutrophil share, the ceiling check and the departure map. Withholds the reading, with the reason, when there is no mix, no neutrophils, or under 90 % of the sites.

Code: `conductor_v3.py` line 125 (26 lines) · arguments: beta, comp · calls: array run 1 · constants read: `BUILD`, `H`

`conductor_v3.py::_ceiling`

The mean β of the neutrophil's methylated identity sites, and whether it has fallen below 0.5, past which Met-A no longer rises with loss.

Code: `conductor_v3.py` line 152 (7 lines) · arguments: x, mu · calls: array run 1

`conductor_v3.py::stage_m_isolated`

Stage M on isolated neutrophils: Met-A against the healthy reference (`stage_m_met_a.read`), the 1 % shift, the ceiling check and the departure map against the healthy neutrophil. Keeps the untared state as `state_own_floor` and marks the reading untared until Stage T.

Code: `conductor_v3.py` line 160 (12 lines) · arguments: beta · calls: not called on the traced runs · constants read: `H`

`conductor_v3.py::stage_mc_cscore`

Stage MC: the C-Score from the departure map, divided by the healthy baseline, with the share of sites more than 3 SD out. With no map, no C-Score and the reason.

Code: `conductor_v3.py` line 173 (12 lines) · arguments: z · calls: array run 1 · files named: `divided by the healthy median clustering of neutrophil_reference_v1_2.json`

`conductor_v3.py::_ref_records`

Turns the same-run references into records (A, neutrophil share, N, C), skipping empty values and the sample itself.

Code: `conductor_v3.py` line 186 (13 lines) · arguments: ref_A, sample_id · calls: array run 2

`conductor_v3.py::stage_t_tare`

Stage T step 2: the median tare of Met-A, the reference spread and the detection limit; untared, with the reason, under 3 references.

Code: `conductor_v3.py` line 200 (16 lines) · arguments: A, ref_A, shift_1pct, sample_id · calls: array run 1 · constants read: `MIN_REFS`

`conductor_v3.py::stage_t_cscore`

The C-Score's same-run tare and its call against the band (above 1.152 a clustered departure).

Code: `conductor_v3.py` line 219 (18 lines) · arguments: cs, ref_A, sample_id · calls: array run 1 · constants read: `CSCORE_COMMISSIONING`, `CSCORE_DEV_BAND`, `MIN_REFS`

`conductor_v3.py::stage_t_selftare_ii`

Stage T step 1: runs `dev_stages.selftare_map` on the array and returns the mapped values with the anchors and maps; unmapped values and a NOT_RUN record if the map file is missing.

Code: `conductor_v3.py` line 238 (9 lines) · arguments: beta · calls: array run 1

`conductor_v3.py::platform_refusal`

Nothing for an EPIC v1 array; otherwise the refusal: another array type from Stage 0, EPIC v2 probe names, or 700,000 probes or fewer.

Code: `conductor_v3.py` line 248 (10 lines) · arguments: beta, array_type · calls: array run 1 · constants read: `ACCEPTED_ARRAY_TYPES`

conductor_v3.py::run_neutrophil

The whole conductor, in the order above. Returns the array reading's records (mix, Met-A, tare, C-Score) and the departure map for the sky.

Code: conductor_v3.py line 259 (50 lines) · arguments: beta, specimen, ref_A, array_type, sample_id · calls: array run 1 · constants read: BUILD, CSCORE_COMMISSIONING, METAA_COMMISSIONING, MIN_NOISE_FRACTION

dev_stages.py::_json

Loads a file from Runtime Matrices/Development once and keeps it.

Code: dev_stages.py line 37 (5 lines) · arguments: name · calls: array run 1 · constants read: DEVDIR, _C

dev_stages.py::anchors

This array's mean β over each of the four fixed-site sets (type I low and high, type II low and high), with how many it measured.

Code: dev_stages.py line 54 (7 lines) · arguments: beta, st · calls: array run 1 · files named: dev_selftare_typeII_EPIC_v1.json

dev_stages.py::selftare_map

Stage T step 1, as described above: maps each design's values by β' = L_r + (β – L)(U_r – L_r)/(U – L) at the sites whose design is recorded and returns the maps' anchors and slopes.

Code: dev_stages.py line 63 (12 lines) · arguments: beta, st · calls: array run 1 · files named: dev_selftare_typeII_EPIC_v1.json

stage_m_met_a.py::_floors

Loads metA_floors_v1_3.json (the 6,000 identity sites and the healthy reference H_ref) once.

Code: stage_m_met_a.py line 29 (4 lines) · arguments: none · calls: array run 1 · constants read: FLOORS, _F

stage_m_met_a.py::_H

The entropy of a beta value in bits, with β clipped to [10■■, 1 – 10■■] so a value of exactly 0 or 1 gives 0, not an error.

Code: stage_m_met_a.py line 34 (3 lines) · arguments: b · calls: array run 7

stage_m_met_a.py::platform_of

The platform from the values themselves: EPIC v2 when any probe name carries the v2 design suffix; else EPIC when more than 700,000 probes, else 450K.

Code: stage_m_met_a.py line 40 (6 lines) · arguments: beta · calls: array run 2 · constants read: _V2_NAME

stage_m_met_a.py::available

The cell types with a healthy reference on each platform that are in scope (neutrophils).

Code: stage_m_met_a.py line 47 (3 lines) · arguments: platform · calls: not called on the traced runs · constants read: CELLS_IN_SCOPE

stage_m_met_a.py::precision

The held-out readings of the reference arrays for one cell type: how many, the share in Normal, the SD and the range.

Code: `stage_m_met_a.py` line 51 (6 lines) · arguments: platform, cell · calls: not called on the traced runs · constants read: `LOO`, `NORMAL`

`stage_m_met_a.py::read`

Met-A on a purified specimen of one cell type: refuses a cell outside scope, a platform without a reference, or under 90 % of the sites; otherwise A = mean H(β) / H_ref with its state and the held-out precision.

Code: `stage_m_met_a.py` line 58 (19 lines) · arguments: beta, cell, platform, specimen · calls: not called on the traced runs · constants read: `CELLS_IN_SCOPE`, `NORMAL`, `SITE_COVERAGE_MIN`

6. Chain v4: the cell shares, IAM-A and the sky: `stage_v4.py`

What it is for. Bringing the instruments commissioned after chain v3 into the one run: Stage D21 (each blood cell's share, from the IAM-Deconvolver against IAM-Atlas v2.1), Stage Q2 (IAM-A on sequencing data) and the sky (a picture of the C-Score's departures). Nothing here is fitted at run time; every reference is a committed, stamped file.

Stage D21's gate. The six-cell blood panel (neutrophils, monocytes, B, CD4 T, CD8 T, NK) is read on the sites that separate those cells in IAM-Atlas v2.1. With fewer than half of those marker sites measured, the shares are withheld and the reason gives the count. The 50 % line is operational, not measured. Shares of neutrophils, monocytes, CD4 T and CD8 T cells carry the commissioned label; B and NK cells are printed as development. The error bars are information, not calibrated intervals.

Stage Q2's order. Count the molecules (from a `.pat` file or an aligned BAM), refuse a file with no qualifying molecules or tiles, run the sample-shape check, then read A against the sample's own reference and band v1. The commissioned label attaches to CD8 T cells (the parent type and its subsets) read by whole-genome bisulfite; every other cell type or chemistry is labeled development.

The sky. The C-Score's departure map, in the C-Score's own genomic order, laid on a HEALPix sphere in NESTED order: consecutive indices fill compact patches, so a run of neighboring departures (what the C-Score scores) shows as a patch and scattered departures as speckle. NSIDE is the smallest power of 2 giving at least one pixel per two sites. Departures are drawn in units of the sample's own spread (median and 1.4826 × MAD), which leaves the C-Score unchanged, because the C-Score is unchanged by shifting or rescaling every departure. The pixel-to-angle conversion is the standard HEALPix NESTED algorithm (Górski et al. 2005), written out here so the chain does not need healpy; it matches healpy to 10■¹■ radians. **Label: standard method** (the HEALPix sphere of cosmology, used as a display). Nothing is scored from the picture.

`stage_v4.py::_ex`

The five-part explanation of a stop, for the cell shares and IAM-A (the same parts as `run_sample._why`).

Code: `stage_v4.py` line 22 (2 lines) · arguments: check, outcome, measured, limit, source, means, todo, provisional · calls: not called on the traced runs

`stage_v4.py::stage_d21`

Stage D21: loads the panel's reference and markers once, counts the markers measured, withholds below half, otherwise runs the Deconvolver and labels each cell's share commissioned or development.

Code: `stage_v4.py` line 27 (20 lines) · arguments: beta, panel · calls: array run 1 · constants read: `MIN_MARKER_SHARE`, `_REF`

`stage_v4.py::count_sequencing`

Counts a `.pat` file (gzipped or plain) on the hg19 tiles, or an aligned BAM against its reference FASTA, with Stage Q2's one counter.

Code: `stage_v4.py` line 49 (5 lines) · arguments: pat, bam, ref_fasta, workers, assembly · calls: bam run 1

`stage_v4.py::stage_q2`

Stage Q2 as above: the refusals, the shape check, the reading A with its predicted value, and the commissioned or development label.

Code: `stage_v4.py` line 55 (26 lines) · arguments: rec, cell, chemistry, source · calls: bam run 1 · constants read: `CD8_TYPES` · files named: `Runtime Matrices/IAM_A_SelfRef/shape_reference_v1.json`

`stage_v4.py::_compress`

Takes every other bit of an integer: the step of the HEALPix algorithm that splits a NESTED pixel index into its x and y coordinates.

Code: `stage_v4.py` line 88 (3 lines) · arguments: v · calls: array run 6

`stage_v4.py::nest_pix2ang`

HEALPix NESTED pixel centers to colatitude and longitude.

Code: `stage_v4.py` line 91 (10 lines) · arguments: nside, ipix · calls: array run 3 · constants read: `JPLL`, `JRLL`

`stage_v4.py::cscore_sky`

Draws the C-Score's departures on the sphere in a Mollweide view and returns a summary (NSIDE, sites, pixels drawn, the share of departures beyond 3). Without a departure map it draws nothing and says why.

Code: `stage_v4.py` line 102 (38 lines) · arguments: z, out_png, title, vlim, readings · calls: array run 1

`stage_v4.py::png_data_uri`

Embeds a figure in the HTML report as a data URI.

Code: `stage_v4.py` line 141 (2 lines) · arguments: path · calls: not called on the traced runs

7. The IAM-Deconvolver: `stage_deconv_v21.py`

What it is for. Given a mixed sample's array values, each cell's share, with a 95 % interval.

Label: standard method (weighted non-negative least squares against reference cell profiles), with IAM-Atlas v2.1 as the reference.

The reference. For each site and cell, the atlas's best value μ and the spread a new person's cell has around it: $sd^2 = ((hi - lo)/3.92)^2 + donor_sd^2 + lab_sd^2 + 0.01^2$. The first term is the uncertainty of the atlas's average (its 95 % range); donor_sd is the person-to-person spread the atlas fit measured; lab_sd is the laboratory-to-laboratory spread, the SD of the fit's own array-laboratory offsets (18 laboratories, 0.0065 when sealed); 0.01^2 is a floor. Version 1 used only the first term and is kept as `SPREAD="mean_only"`.

The markers. For each cell, the 50 sites where it sits furthest above, and the 50 furthest below, its nearest other cell in the panel, in units of their combined spread: $z = |\mu_c - \mu_{nearest}| / \sqrt{(sd_c^2 + sd_{nearest}^2)}$.

The solver. Non-negative least squares on the markers, each site weighted by $1 / \sqrt{(\Sigma_k f_k^2 sd_k^2 + lab^2)}$, starting from equal shares and refined over three passes. The shares are scaled to sum to 1 and the raw sum is reported.

The intervals. 200 refits, each on resampled marker sites and on cell profiles redrawn from the atlas's spread (a new person's cells are not the average cell). The 2.5th and 97.5th percentiles give the interval. On held-out data they covered the truth 78 % of the time, below the 95 % their name implies: they are information, not calibrated intervals (see below).

Built from, tested on. Built from IAM-Atlas v2.1 alone. Tested by planted mixtures drawn from the atlas's own spread with a laboratory shift (the file's `planted` test), then by the sealed real test: 200 mixtures of held-out purified arrays from E-MTAB-2145, a laboratory the atlas never saw. Commissioned 2026-10-17 on the shares of neutrophils, monocytes, CD4 T and CD8 T cells: mean share error 0.016, every number reproduced from a fresh clone (on AWS

2026-10-18 and on GitHub 2026-10-20). Both sealed tests carried two bars: the mean share error at most 0.03 (met: 0.029 for version 1, 0.016 for version 2), and the true share inside the 95 % interval at least 80 % of the time (not met: 0.42, then 0.78). After both scores, the author's limits study (2026-10-17) held the tool to its job, the share, so the interval bar no longer gates it; the commissioning note discloses that the decision came after the scores, and the scores stand unchanged. The intervals are therefore printed as information, labeled not calibrated.

What it reads outside its own folder. The laboratory spread comes from `doors/data/DEV_ATLAS_V21_01/transfer_v21_final.json`, a file in a test-record folder. It is a runtime input and belongs in the chain's references (a to-do item of this Decoder).

How to break it. Read a pure array of one cell: its own share should be at least 0.80. Plant a 5 % share of one cell into a mixture: its interval should exclude 0 in most mixtures. Remove half the markers: the shares must be withheld.

`stage_deconv_v21.py::reference`

Reads μ, its range and the donor spread for the named cells from the atlas's compact parquet files (only those columns; 65,535 marks a missing value) and builds sd as above.

Code: `stage_deconv_v21.py` line 26 (23 lines) · arguments: cells, atlas, spread · calls: array run 1 · constants read: `ATLAS`, `SPREAD` · files named: `import pyarrow.parquet`, `sites.parquet`

`stage_deconv_v21.py::markers`

The marker sites for the panel, as above.

Code: `stage_deconv_v21.py` line 49 (8 lines) · arguments: R, per_cell · calls: array run 1 · constants read: `PER_CELL`

`stage_deconv_v21.py::_solve`

The three-pass weighted non-negative least squares.

Code: `stage_deconv_v21.py` line 57 (5 lines) · arguments: y, A, S, lab_sd, passes · calls: array run 201

`stage_deconv_v21.py::deconvolve`

The shares, raw sum, 95 % intervals and sites used for one sample.

Code: `stage_deconv_v21.py` line 62 (14 lines) · arguments: beta, R, M, lab_sd, boot, seed · calls: array run 1 · constants read: `BOOT`, `LAB_SD`, `SEED`

`stage_deconv_v21.py::sample`

Inside the planted test only: builds a planted array, each cell's DNA drawn from the atlas's donor spread, mixed in known shares, plus a laboratory shift and array noise.

Code: `stage_deconv_v21.py` line 93 (2 lines) · arguments: f, lab · calls: not called on the traced runs

`stage_deconv_v21.py::sample_check`

For the second sealed test: a pure array passes when it reads at least 0.80 of its own cell. Failing arrays are listed and not mixed.

Code: `stage_deconv_v21.py` line 170 (2 lines) · arguments: x, cell, R, M · calls: not called on the traced runs · constants read: `SELF_MIN`

8. IAM-Atlas v2.1's commissioning rules: `stage_atlas_commission.py`

What it is for. The scorers IAM-Atlas v2.1 was commissioned by, and the table that says which uses of the atlas are commissioned. The chain loads it for the laboratory spread (`lab_sd`) and for that table.

Label: standard method (a posterior predictive interval for a new sample from a new laboratory, scored by coverage).

The uses. U1, reference profiles: a healthy cell from any laboratory reads inside the atlas's range for that cell. Commissioned for cortical neurons, lung bronchus epithelium and naive CD4 T cells on 450K. U2a, blind sites: commissioned for colon and lung. U3, composition: development. A use not listed is development wherever it appears.

The bars (as amended and sealed 2026-10-16, before the atlas existed). U1: at least 95 % of a cell's healthy held-out samples, all laboratories pooled, cover 80 % to 97 % of their sites inside the atlas's 90 % predictive range. U2: a tissue passes when its epithelium is assessed and at least 95 % of its assessed cells reach a share of 0.90. U3: neutrophil mean absolute error ≤ 0.02 and at least 95 % of samples within 0.05. A sample the chain withholds is listed, not scored.

`stage_atlas_commission.py::lab_sd`

The SD of the atlas fit's array-laboratory offsets, read from `transfer_v21_final.json` at run time, never typed.

Code: `stage_atlas_commission.py` line 17 (4 lines) · arguments: terms · calls: array run 1 · constants read: `TERMS`

`stage_atlas_commission.py::predictive_interval`

The 90 % predictive range for a new sample from a new laboratory. Arrays: N(μ, sd² + lab² + floor). Sequencing: the array value carried through the platform map, plus binomial counting noise at the sample's coverage, the laboratory scatter and the floor.

Code: `stage_atlas_commission.py` line 21 (11 lines) · arguments: draws, sd_draws, kind, g, cov, scatter, level, seed, lab · calls: not called on the traced runs · constants read: `FLOOR`, `NOISE`, `RNG_SEED`

`stage_atlas_commission.py::u1_coverage`

The share of a sample's sites that fall inside the predictive range.

Code: `stage_atlas_commission.py` line 32 (2 lines) · arguments: lo, hi, y · calls: not called on the traced runs

`stage_atlas_commission.py::u1_verdict`

U1's verdict for one cell: commissioned when at least 95 % of its scored samples have coverage between 0.80 and 0.97.

Code: `stage_atlas_commission.py` line 34 (6 lines) · arguments: cov_by_sample, withheld, lo_bar, hi_bar · calls: not called on the traced runs · constants read: `SHARE`

`stage_atlas_commission.py::u2_share`

The share of a tissue set's sites within tolerance of the anchor.

Code: `stage_atlas_commission.py` line 40 (2 lines) · arguments: set_mean, anchor, tol · calls: not called on the traced runs

`stage_atlas_commission.py::u2_verdict`

U2's verdict for one tissue.

Code: `stage_atlas_commission.py` line 42 (4 lines) · arguments: shares, bar, epithelium · calls: not called on the traced runs · constants read: `SHARE`

`stage_atlas_commission.py::u3_neutrophil`

U3's neutrophil error: the mean absolute error, the share within 0.05, the largest error, and whether both bars are met.

Code: `stage_atlas_commission.py` line 46 (4 lines) · arguments: est, truth, withheld_mask · calls: not called on the traced runs · constants read: SHARE

`stage_atlas_commission.py::atlas_v21_use_status`

Commissioned or development for one use of the atlas, read from the table above.

Code: `stage_atlas_commission.py` line 56 (6 lines) · arguments: use, key · calls: not called on the traced runs · constants read: `ATLAS_V21_COMMISSIONING`

9. IAM-A, each sample against its own reference: `stage_q2_selfref.py`

What it is for. Reading, from single DNA molecules, how well a cell holds its methylation compared with what its own state predicts. No other person enters the reading.

Label: from IAM's physics. At a held site, writing (on SAM) and erasing (on oxygen and 2-oxoglutarate) are both running. Each interval a mark is lost with probability f and restored with probability g, so the share of held sites that are wrong at any moment settles at the **held error**

ε = f / (f + g), and the holding energy is E_hold = kT ln[(1 – ε)/ε].

The balance is set by the ratio, not by how often the cell divides; division only sets how fast the balance is reached. Measured this way on 56 healthy cell types, E_hold is 3.08 kT per interior opportunity (`holding_energy_q2.json`).

What is counted (one counter, two readers of the same molecules).

- **The held error ε.** On molecules with at least 6 CpG calls and at least 80 % methylated (a held run), the isolated unmethylated

interior calls (a T between two Cs) over the interior calls. The first and last call of a molecule are never counted, because a hole there cannot be told from the end of the run.

- **The loose share L.** The genome is cut into tiles of 2,000 consecutive CpGs. A tile is loose when its molecules' mean methylation is

below 0.70. L is the share of tiles (each with at least 200 calls) that are loose: how much of its genome this cell has let go.

- **No molecule is used twice.** Molecules alternate: even ones are counted for L, odd ones for ε. The reference (from L) and the

quantity it judges (ε) never share a molecule. In a `.pat` file the alternation is by line, and a line carries every identical molecule with its count, so all copies of one pattern go to the same side.

The self-reference line. Across healthy cell types, ε rises in a straight line with L:

ε_predicted = firm + slope × L, with firm = 0.04026 and slope = 0.03462 (`selfref_line_v1.json`).

This line was measured once on 153 healthy samples of 56 cell types (a public whole-genome bisulfite atlas of purified human cells, GSE186458), counted by this same counter. A Theil–Sen fit was used, the median of pairwise slopes, which is robust to outlying cell types. The median distance of a cell type from the line is 0.0025. The line is stored on the raw axis this counter writes; there is no scale factor anywhere. What the intercept means physically is the subject of a recount (to-do step 14); the reading uses it only as a measured constant.

The reading. IAM-A = ε measured / ε predicted at the sample's own L. 1 is on the healthy line; above 1 is more holes than its own state predicts. **Band v1: 0.7787 to 1.0551**, the 2.5th to 97.5th percentile of 156 pooled healthy readings.

Built from, tested on. The line, the shape reference and 153 of the band's 156 readings come from the same 153 healthy samples (GSE186458); the band's other 3 readings are from a second laboratory (GSE49627). So the band describes the healthy spread inside the set the line was measured on. What commissioned IAM-A was the sealed grand test on held-out samples from a third study: healthy CD8 T cells, of which 8 were read (5 declined by the shape check) and 7 read Normal (bar: at least 75 % inside, at least 8 read).

The sample-shape check (a gate). A sample is read only if the 10th percentile of its tile means is at most 0.0543 below the lowest healthy sample of its own cell type. The margin is the 90th percentile, over the 53 cell types with at least 2 healthy samples, of the spread of that percentile within a type. It is computed from the reference, never typed. A cell type with no healthy reference is not read.

The stamp. Every output this stage writes carries a chain stamp: the stage, a SHA-256 of the chain's code, a SHA-256 of the result, its inputs, the Master Testing Manifest rows it belongs to, and the time. The line, band and shape reference are refused if they were edited after the chain wrote them, because the result's hash no longer matches its stamp.

How to break it. Plant isolated holes into held runs of a constructed BAM: ε and IAM-A must rise (release check V7 does this through the one command). Hand it a cell type with no healthy reference: it must not read. Edit one number in `band_v1.json`: the chain must refuse it.

`stage_q2_selfref.py::chain_sha`

The SHA-256 over every `.py` file of the chain folder, names and contents, in sorted order.

Code: `stage_q2_selfref.py` line 28 (4 lines) · arguments: none · calls: not called on the traced runs · constants read: `HERE`

`stage_q2_selfref.py::_canon`

A result in canonical JSON (sorted keys, no spaces), so the same result always hashes the same.

Code: `stage_q2_selfref.py` line 32 (1 lines) · arguments: obj · calls: bam run 4

`stage_q2_selfref.py::payload_sha`

The SHA-256 of a result without its stamp.

Code: `stage_q2_selfref.py` line 33 (1 lines) · arguments: obj · calls: bam run 4

`stage_q2_selfref.py::file_sha`

The SHA-256 of a file, read in 16 MB pieces.

Code: `stage_q2_selfref.py` line 34 (5 lines) · arguments: path, chunk · calls: not called on the traced runs

`stage_q2_selfref.py::_rows`

The manifest rows a result belongs to, given directly or through the `IAM_MANIFEST` environment variable.

Code: `stage_q2_selfref.py` line 44 (3 lines) · arguments: manifest · calls: not called on the traced runs

`stage_q2_selfref.py::stamp`

Attaches the chain stamp to a result, hashing it exactly as it will be stored (JSON turns number keys into text).

Code: `stage_q2_selfref.py` line 47 (8 lines) · arguments: obj, inputs, manifest · calls: not called on the traced runs · constants read: `STAGE`

`stage_q2_selfref.py::write_json`

Writes a stamped result and reads it back to confirm the stored file verifies.

Code: `stage_q2_selfref.py` line 55 (6 lines) · arguments: obj, path, inputs, manifest · calls: not called on the traced runs

`stage_q2_selfref.py::write_csv`

Writes a table and its stamp beside it, as `<file>.stamp.json`.

Code: `stage_q2_selfref.py` line 61 (4 lines) · arguments: df, path, inputs, manifest · calls: not called on the traced runs · constants read: `STAGE`

`stage_q2_selfref.py::hg19_tiles`

The hg19 tiles of 2,000 consecutive CpGs on each autosome, in CpG-index coordinates; a partial last tile is dropped.

Code: `stage_q2_selfref.py` line 67 (7 lines) · arguments: none · calls: not called on the traced runs · constants read: `HG19_CPGS`

`stage_q2_selfref.py::_finish`

Turns tile counts and hole counts into the record: L, the number of tiles, ε and its opportunities, and the 10th, 50th and 90th percentiles of the tile means for the shape check.

Code: `stage_q2_selfref.py` line 74 (9 lines) · arguments: tile_calls, e_iso, e_opp, extra · calls: bam run 1

`stage_q2_selfref.py::count_pat`

The `.pat` reader: even lines are added to their tile; odd lines that qualify as held runs add their interior calls and isolated holes.

Code: `stage_q2_selfref.py` line 83 (16 lines) · arguments: lines, tiles · calls: not called on the traced runs

`stage_q2_selfref.py::_cpg_index`

The positions of every CpG in a chromosome's sequence.

Code: `stage_q2_selfref.py` line 102 (2 lines) · arguments: seq · calls: not called on the traced runs

`stage_q2_selfref.py::_calls_of`

One aligned read's methylation calls at CpGs, from the Bismark XM tag or, for gemBS, from the read against the reference. Skips the first and last 3 aligned bases. Also counts non-CpG conversion (a check on bisulfite conversion) and, on every 20th read, A/T mismatches (a check on sequencing error).

Code: `stage_q2_selfref.py` line 104 (27 lines) · arguments: r, ref, ref0, sub · calls: not called on the traced runs

`stage_q2_selfref.py::_merge`

Merges the calls of the two reads of a pair into one molecule; a CpG on which they disagree is dropped.

Code: `stage_q2_selfref.py` line 131 (6 lines) · arguments: a, b · calls: not called on the traced runs

`stage_q2_selfref.py::count_bam_region`

Counts one 10 Mb region of an alignment file: primary, mapped, non-duplicate reads with mapping quality at least 10, pairs merged, then the same alternation and the same two counts as the `.pat` reader. Also counts calls per qualifying molecule for the response curve.

Code: `stage_q2_selfref.py` line 137 (34 lines) · arguments: args · calls: not called on the traced runs

`stage_q2_selfref.py::count_bam_region.done`

Counts one finished molecule: even molecules to their tile, odd held runs to ε.

Code: `stage_q2_selfref.py` line 144 (14 lines) · arguments: calls · calls: not called on the traced runs

`stage_q2_selfref.py::combine_bam`

Adds the regions' counts into one record, with the conversion-failure and substitution rates.

Code: `stage_q2_selfref.py` line 171 (8 lines) · arguments: parts, assembly · calls: bam run 1

`stage_q2_selfref.py::count_bam`

Splits the autosomes into 10 Mb regions and counts them in parallel.

Code: `stage_q2_selfref.py` line 179 (5 lines) · arguments: bam, fa, workers, assembly, chroms, span · calls: bam run 1 · constants read: `AUTOSOMES`

`stage_q2_selfref.py::build_line`

How the line was built: per cell type, the median L and the opportunity-weighted ε, then a Theil–Sen line across cell types. Not called at run time; the line is read from its frozen file.

Code: `stage_q2_selfref.py` line 186 (10 lines) · arguments: per_sample · calls: not called on the traced runs

`stage_q2_selfref.py::line`

Loads the frozen line and refuses it if it was edited after the chain wrote it.

Code: `stage_q2_selfref.py` line 196 (6 lines) · arguments: version · calls: bam run 1 · constants read: `LINE_DIR`

`stage_q2_selfref.py::predict`

ε predicted at a loose share: firm + slope × L.

Code: `stage_q2_selfref.py` line 202 (2 lines) · arguments: loose_share, version · calls: not called on the traced runs

`stage_q2_selfref.py::residual`

A count against the line: measured ε, predicted ε, and their difference.

Code: `stage_q2_selfref.py` line 204 (5 lines) · arguments: rec, version · calls: bam run 1

`stage_q2_selfref.py::build_shape_reference`

How the shape reference was built: per cell type, the lowest healthy 10th percentile, and the margin. Not called at run time.

Code: `stage_q2_selfref.py` line 215 (8 lines) · arguments: records · calls: not called on the traced runs · constants read: `SHAPE_Q`

`stage_q2_selfref.py::shape_reference`

Loads the frozen shape reference and refuses it if edited.

Code: `stage_q2_selfref.py` line 223 (6 lines) · arguments: version · calls: bam run 2 · constants read: `LINE_DIR`

`stage_q2_selfref.py::shape_check`

The sample-shape gate described above, with the measured percentile, the limit and the reason.

Code: `stage_q2_selfref.py` line 229 (9 lines) · arguments: rec, cell, version, ref · calls: bam run 1

`stage_q2_selfref.py::reading`

IAM-A = measured ε / predicted ε.

Code: `stage_q2_selfref.py` line 240 (4 lines) · arguments: rec, version · calls: bam run 1

`stage_q2_selfref.py::build_band`

How band v1 was built: the 2.5th to 97.5th percentile of pooled healthy readings. Not called at run time.

Code: `stage_q2_selfref.py` line 244 (6 lines) · arguments: healthy, version · calls: not called on the traced runs

`stage_q2_selfref.py::band`

Loads band v1 and refuses it if edited.

Code: `stage_q2_selfref.py` line 254 (2 lines) · arguments: version · calls: bam run 1 · constants read: `LINE_DIR`

`stage_q2_selfref.py::grand_verdict`

The sealed grand test's scorer: each healthy sample shape-checked, each read sample inside or outside the band; met when at least 75 % of at least 8 read samples are inside.

Code: `stage_q2_selfref.py` line 258 (12 lines) · arguments: records, version, B, ref · calls: not called on the traced runs · constants read: `GRAND_MIN_READ`, `GRAND_SHARE`

10. Stage Q0, intake for sequencing files: `stage_q0_intake.py`

What it is for. Stage 0 for a `.pat` file: an array cannot reach Met-A without Stage 0, and a `.pat` file cannot reach IAM-A without Stage Q0. Each check stops the file with a named reason, and every measured value is recorded whether or not it stops the file.

Label: standard method (file integrity, genome build, coverage and conversion checks, as in the ENCODE whole-genome bisulfite standard).

Check	What it measures	Stops the file when	Limit and source
Q0.7 specimen	the cell type	on the chain v4 route, the cell type has no healthy shape reference	rule
Q0.1 readable	the file decompresses to its end; every line has 4 fields (chromosome, CpG index, a pattern of C, T and dots, a count)	any malformed line or a truncated file	rule
Q0.2 build	every line's CpG index lies inside its chromosome's range in the hg19 CpG dictionary (`hg19_cpg_chrom_ranges.json`)	any line outside	rule
Q0.4 whole genome	molecules on every autosome	an autosome has none	rule
Q0.4 coverage	interior calls on molecules with at least 6 calls	never (recorded)	—
Q0.5 read length	the share of molecules with at least 6 calls	not set: recorded only	to be set on healthy files
Q0.3 conversion	the bisulfite conversion rate from the alignment record, when one is given	below 0.98	the ENCODE whole-genome bisulfite standard
Q0.6 duplicates	the duplicate fraction from the alignment record, when given	not set: recorded only	to be set on healthy files

Not run on a BAM. Q0's file checks are written for the `.pat` format. On the `--bam` route the intake record says so and why, and Stage Q2's counter records the bisulfite conversion failure (from non-CpG calls) and the substitution rate on A and T; the sample-shape check refuses a cell type with no healthy reference.

`stage_q0_intake.py::_ranges`

Loads the hg19 CpG-index range of each chromosome, if the file is present.

Code: `stage_q0_intake.py` line 35 (3 lines) · arguments: none · calls: not called on the traced runs · constants read: `RANGES`

`stage_q0_intake.py::scan_pat`

One pass over the file, gzipped (including multi-member bgzip) or plain: completeness, malformed lines, molecules per chromosome, calls, molecules with at least 6 calls, interior opportunities, and lines outside their chromosome's range.

Code: `stage_q0_intake.py` line 40 (36 lines) · arguments: path · calls: not called on the traced runs · constants read: `MIN_CALLS` · files named: `hg19_cpg_chrom_ranges.json`

`stage_q0_intake.py::scan_pat.line`

Parses and counts one line.

Code: `stage_q0_intake.py` line 45 (11 lines) · arguments: l · calls: not called on the traced runs · constants read: `MIN_CALLS`

`stage_q0_intake.py::intake`

Runs the checks in the table and returns the record with each check's value, rule and result, and the decision: stop at the first refusal, or proceed.

Code: `stage_q0_intake.py` line 78 (49 lines) · arguments: path, specimen, cell, alignment_qc, selfref_cells · calls: not called on the traced runs · constants read: `ACCEPTED`, `AUTOSOMES`, `MIN_CALLS`, `Q0_LIMITS`

`stage_q0_intake.py::intake.chk`

Records one check and, when it stops the file, the first refusal's code and reason, with its five-part explanation.

Code: `stage_q0_intake.py` line 82 (10 lines) · arguments: step, ok, value, rule, code, stop · calls: not called on the traced runs

11. Group-A, has a whole group moved: `stage_groupa.py`

What it is for. One person's whole-blood Met-A cannot show a 1 % change: healthy people differ by about 2 % after the tare. The average of a group can. Group-A compares two groups read in one series.

Label: standard method (two tared means, Welch's standard error), applied to one physical reading per person, each against its own expected value for that person's cell mix.

How it reads. Each person is read by the one command, unchanged. Each reading is tared against healthy arrays run the same way: the median of the healthy references on the same slide when the slide holds at least 3, else of the same series. A sample is never its own reference. Then the two groups' tared means are compared:

difference d = mean■ – mean■, standard error SE = √(s■²/n■ + s■²/n■), t = d / SE.

t	≥ 2 is a group that moved.	t	< 2 is no difference found, and the smallest difference the design could have found (2 × SE) is

printed. Nothing is fitted.

Built from, tested on. Sealed before any array was opened, then scored once on 278 healthy controls of a laboratory no instrument had read (GSE318173): 263 read; women and men 0.36 % apart at 1.2 standard errors; a 1 % loss planted into half of them found at 7.3 standard errors. Commissioned 2026-10-20 for groups of whole bloods run in one series, EPIC v1. Release check V10 reproduces bars 2 and 3 to every printed digit from the committed rows. Groups in any other specimen are read and labeled preliminary.

Why one series. Across laboratories, untared readings differ by several percent. Two groups from two laboratories would differ by their laboratories.

`stage_groupa.py::tare`

Each reading over the median of its healthy references (same slide when there are at least 3 there, else the series), excluding itself; records which scope was used.

Code: `stage_groupa.py` line 28 (22 lines) · arguments: r, healthy · calls: not called on the traced runs

`stage_groupa.py::welch`

The two groups' means, counts, difference, standard error and t.

Code: `stage_groupa.py` line 51 (4 lines) · arguments: a, b · calls: not called on the traced runs

`stage_groupa.py::compare`

Tares, compares and calls the result, with the smallest findable difference and the commissioning label.

Code: `stage_groupa.py` line 56 (17 lines) · arguments: rows, group_col, a, b, healthy, specimen, platform · calls: not called on the traced runs · constants read: `PRELIMINARY`

`stage_groupa.py::collect`

Builds one row per person from the run's report bundles and the researcher's sample list (id, slide, group column, optional healthy_reference).

Code: `stage_groupa.py` line 74 (8 lines) · arguments: bundle_dir, samples_csv, out_csv · calls: not called on the traced runs · files named: `_bundle.json`

12. The retired position reading, Stage Q: `stage_q_iam_a.py`

Retired. IAM-A was first read against a frozen position P per cell type, measured on purified granulocytes. That reading was replaced on 2026-10-17 by Stage Q2, each sample against its own reference. The one command runs Stage Q only with `--site-table` or `--dev-iama-position`, and labels it development. It is kept so earlier records can be reproduced. Its text still uses the retired term "copy error"; what it counts is the held error described in section 9.

`stage_q_iam_a.py::positions`

Loads the frozen positions file (`iama_positions_v2.json`).

Code: `stage_q_iam_a.py` line 35 (4 lines) · arguments: none · calls: not called on the traced runs · constants read: `POS`, `_P`

`stage_q_iam_a.py::_H`

The entropy of a rate in bits.

Code: `stage_q_iam_a.py` line 40 (1 lines) · arguments: e · calls: not called on the traced runs

`stage_q_iam_a.py::_pat_lines`

Streams the lines of a `.pat` file, gzipped or plain, optionally cut at a byte count (the retired version 1 read only part of each file).

Code: `stage_q_iam_a.py` line 42 (27 lines) · arguments: path, max_bytes, chunk · calls: not called on the traced runs

`stage_q_iam_a.py::pat_site_table`

The per-site table (opportunities and holes in two halves of the molecules) from a `.pat` file.

Code: `stage_q_iam_a.py` line 72 (22 lines) · arguments: path, max_bytes · calls: not called on the traced runs · constants read: `PAT_PIPELINE`

`stage_q_iam_a.py::_genomic_order`

Sorts a site table by chromosome and CpG index.

Code: `stage_q_iam_a.py` line 95 (8 lines) · arguments: T · calls: not called on the traced runs

`stage_q_iam_a.py::cscore`

The retired IAM-A clustering score over the site table, pooled and per half.

Code: `stage_q_iam_a.py` line 105 (14 lines) · arguments: site_table, eps, halves · calls: not called on the traced runs · constants read: `CSCORE_BLOCK_SITES`, `CSCORE_MIN_BLOCKS`

`stage_q_iam_a.py::cscore.one`

The clustering score of one table.

Code: `stage_q_iam_a.py` line 108 (8 lines) · arguments: o, k · calls: not called on the traced runs · constants read: `CSCORE_BLOCK_SITES`, `CSCORE_MIN_BLOCKS`

`stage_q_iam_a.py::read`

The retired reading, IAM-A = H(ε) / (P × H(ε■)), refused for a pipeline other than the one P was measured on, or for part of a file.

Code: `stage_q_iam_a.py` line 121 (27 lines) · arguments: site_table, cell, pipeline, mask, allow_partial · calls: not called on the traced runs · constants read: `IAMA_BAND_STATE`, `MIN_HALF_OPPORTUNITIES`, `MIN_OPPORTUNITIES`

`stage_q_iam_a.py::tare`

The retired reading's same-run tare.

Code: `stage_q_iam_a.py` line 152 (22 lines) · arguments: rec, refs, sample_id · calls: not called on the traced runs · constants read: `IAMA_BAND_STATE`, `MIN_REFS`

13. Two draws of one person, Stage 12b: `serial_mode.py`

What it is for. Comparing two draws of the same person. `run_sample.py` calls two of these functions when `--prior-betas` is given; the rest are tools.

Label: standard method.

`serial_mode.py::check_same_person`

Refuses unless the two draws carry the same identifier hash, array type and pipeline.

Code: `serial_mode.py` line 7 (8 lines) · arguments: now, prior · calls: not called on the traced runs

`serial_mode.py::delta_cells`

The change in each cell's A for cells present in both draws; cells that appeared or disappeared are listed, not differenced. Reads the per-cell layout of an older bundle (`cells_all`); a tool, not called by the one command.

Code: `serial_mode.py` line 16 (16 lines) · arguments: now, prior, floor · calls: not called on the traced runs

`serial_mode.py::delta_sky`

β now minus β before, at the sites both draws measured. No expectation, no composition.

Code: `serial_mode.py` line 33 (8 lines) · arguments: beta_now, beta_prior, floor_beta · calls: not called on the traced runs

`serial_mode.py::snp_noise_floor_A`

The two arrays' own noise (Stage 1's SNP-probe model, $\sigma^2(\beta) = a + b \cdot \beta(1 - \beta)$) carried through to A over the cell's identity sites: $\Delta A \approx 2.576 \times |dH/d\beta| \times \sqrt{(var_now + var_prior)} / H_ref$ at the mean β, the 99 % two-sided change the noise alone could make. A lower bound on the smallest change that means anything. It reads the identity sites from an older bundle layout (`loci` of a per-cell entry) that chain v4 bundles do not carry; it is a tool, not a stage.

Code: `serial_mode.py` line 42 (13 lines) · arguments: pci_entry, beta_now, beta_prior, snp_now, snp_prior, hmin · calls: not called on the traced runs

`serial_mode.py::snp_noise_floor_A.var_mean`

For one array, the variance of the mean β over the identity sites from its noise model, $\Sigma(a + b \cdot \beta(1 - \beta)) / n^2$, and the mean β.

Code: `serial_mode.py` line 49 (3 lines) · arguments: beta, snp · calls: not called on the traced runs

`serial_mode.py::trajectory`

A person's readings in draw order with their dates and the step between draws: numbers in order, no fit, no slope. Reads the same older per-cell layout as `delta_cells`.

Code: `serial_mode.py` line 56 (11 lines) · arguments: bundles · calls: not called on the traced runs

14. The report: `report_v4.py`, `report_v3.py`, `report_figs_v4.py`, `report_pdf_v4.py`

What it is for. Writing what the stages measured, and nothing they did not. One self-contained HTML page per sample, and the same report as two PDFs, built from the same rows and pictures so the formats cannot disagree.

Label: standard method (presentation). The report computes no reading. Every number it prints comes from the run's bundle or from a committed reference file.

Its parts, in page order.

1. **The researcher view**: the reading at a glance (each instrument, the question it answers, its reading, its state and its

commissioning status); AstroGenetics, the physics behind each reading with this run's own numbers; the cell shares; Met-A on its gauge; the C-Score with its sky and the overlay of the sample on its healthy reference; IAM-A on its gauge.

2. **The plain view**: one gauge and one sentence per reading, and the sky.
3. **The technical record** (`report_v3.build`): every stage's record, the red flags, the safeguards, troubleshooting, and the

development toolkit table.

The gauge (`report_figs_v4.gauge_png`). Left end the floor (derived), the Normal band around healthy 1 (measured), right end saturation, every identity site fully scrambled (derived). Every mark is read from `Runtime Matrices/Gauge/cell_gauge_marks_v1.json`; nothing is typed in the drawing code. On IAM-A's scale healthy sits close to the floor, so a magnified strip shows the band. A withheld state draws no gauge at all. The C-Score has no derived ends, because it is a clustering ratio: its bar runs from scattered to clustered, with its band.

Commissioned or not, on every line. A commissioned reading carries its record. Anything else is still shown and says so: "This score is preliminary until [instrument] is officially commissioned on [cell]" (`report_v4.untested`).

Red flags (`report_v3.red_flags`): STOP (refused), FLAG (a commissioned call such as a clustered departure), WITHHELD (a reading not given, with its reason), CAUTION and NOTE. They are written into the bundle as well as the page.

Safeguards (`report_v3.safeguards`), each PASS, FAIL or NOT_RUN on this run:

- **Rendered-claim scan:** no disease, cohort or population words in the report's prose.
- **Formula self-test:** H(0.5) = 1, H(0.25) = 0.81128, H(10■■) under 3 × 10■■.
- **Anchors**, **deconvolver conformance** and **atlas separability.**

How to break it. Write a disease word into a reading's text: the claim scan must fail. Remove a mark from the gauge file: the gauge must not draw.

`report_v4.py::_uri`

A figure file as a data URI, so the page is self-contained.

Code: `MethylPhys_Interface/report_v4.py` line 38 (1 lines) · arguments: path · calls: array run 6, bam run 2

`report_v4.py::_H`

The entropy of a rate in bits, for the physics rows.

Code: `MethylPhys_Interface/report_v4.py` line 39 (1 lines) · arguments: p · calls: bam run 2

`report_v4.py::_sci`

A number in HTML scientific notation (×10■).

Code: `MethylPhys_Interface/report_v4.py` line 40 (2 lines) · arguments: v · calls: bam run 2

`report_v4.py::_marks`

Loads the gauge marks file.

Code: `MethylPhys_Interface/report_v4.py` line 42 (1 lines) · arguments: none · calls: array run 3, bam run 3 · constants read: `CH` · files named: `cell_gauge_marks_v1.json`

`report_v4.py::_href`

Met-A's healthy reference in bits per site (0.33026), read from `metA_floors_v1_3.json`, never typed.

Code: `MethylPhys_Interface/report_v4.py` line 45 (2 lines) · arguments: none · calls: not called on the traced runs · files named: `metA_floors_v1_3.json`

`report_v4.py::cellname`

A cell type's display name.

Code: `MethylPhys_Interface/report_v4.py` line 47 (2 lines) · arguments: c · calls: array run 30, bam run 1 · constants read: `_NAMES`

`report_v4.py::untested`

The line printed beside a reading outside what its instrument was commissioned on.

Code: `MethylPhys_Interface/report_v4.py` line 49 (5 lines) · arguments: what, inst, shown, noun · calls: array run 7 · constants read: `_NAMES`

`report_v4.py::chip`

The colored status label: commissioned, refused or withheld, not run, or development.

Code: `MethylPhys_Interface/report_v4.py` line 54 (4 lines) · arguments: status · calls: array run 12, bam run 5 · constants read: `E`

`report_v4.py::_glance`

The at-a-glance table as HTML.

Code: `MethylPhys_Interface/report_v4.py` line 59 (3 lines) · arguments: o · calls: array run 1, bam run 1 · constants read: `E`

`report_v4.py::glance_rows`

The at-a-glance rows, from the bundle: the Deconvolver's top three shares, Met-A (tared when it can be), the C-Score, and IAM-A, each with its state and status. The PDFs use the same rows.

Code: `MethylPhys_Interface/report_v4.py` line 63 (30 lines) · arguments: o · calls: array run 2, bam run 2

`report_v4.py::_shares`

The cell-shares section: each share with its 95 % interval and status, or the reason they were withheld.

Code: `MethylPhys_Interface/report_v4.py` line 94 (12 lines) · arguments: o · calls: array run 1, bam run 1 · constants read: `E`

`report_v4.py::_meta`

The Met-A section: its gauge (tared reading; an untared whole-blood reading has no gauge position) and its numbers.

Code: `MethylPhys_Interface/report_v4.py` line 107 (9 lines) · arguments: o · calls: array run 1, bam run 1 · constants read: `E`

`report_v4.py::_cscore`

The C-Score section: its bar, its sky and the overlay.

Code: `MethylPhys_Interface/report_v4.py` line 117 (9 lines) · arguments: o · calls: array run 1, bam run 1 · constants read: `E`

`report_v4.py::_iama`

The IAM-A section: its gauge, or the refusal.

Code: `MethylPhys_Interface/report_v4.py` line 127 (10 lines) · arguments: o · calls: array run 1, bam run 1 · constants read: `E`

`report_v4.py::_figs`

Draws this sample's gauge and C-Score pictures beside the report.

Code: `MethylPhys_Interface/report_v4.py` line 139 (17 lines) · arguments: o, base · calls: array run 1, bam run 1

`report_v4.py::_physics`

The AstroGenetics section as HTML.

Code: `MethylPhys_Interface/report_v4.py` line 161 (6 lines) · arguments: o · calls: array run 1, bam run 1 · constants read: `E`, `PHYS_INTRO`

`report_v4.py::physics_rows`

Each reading set in the physics it rests on, with this run's numbers. IAM-A: ε measured, ε predicted at the sample's own L, saturation 0.5/ε_pred, and the floor 1/(1 + e^M) as a fraction of ε_pred. Met-A: the mean entropy over the identity sites against its expectation, what healthy 1 is on this run (the median of the same-run references, or the committed healthy reference), and on isolated neutrophils the reading in bits per site. The C-Score: the clustering ratio and its band. The 0.330263 bits per site it prints is typed in this function rather than read from `metA_floors_v1_3.json` (a fix on the Decoder's list).

Code: `MethylPhys_Interface/report_v4.py` line 168 (32 lines) · arguments: o · calls: array run 2, bam run 2 · constants read: `E`

`report_v4.py::_skypair`

The overlay of the sample on its healthy reference, with how sure the healthy reference is at those sites.

Code: `MethylPhys_Interface/report_v4.py` line 202 (8 lines) · arguments: o · calls: array run 1 · constants read: `SKY_CAP`

`report_v4.py::_plain`

The plain view: one gauge and one sentence per reading.

Code: `MethylPhys_Interface/report_v4.py` line 211 (19 lines) · arguments: o, F · calls: array run 1, bam run 1 · constants read: `E`

`report_v4.py::build`

Builds the whole page: the technical record from `report_v3.build`, the figures, both views, and the two PDFs.

Code: `MethylPhys_Interface/report_v4.py` line 231 (27 lines) · arguments: o, out, sid · calls: array run 1, bam run 1 · constants read: `CSS`, `E`

`report_v3.py::_gauge`

The gauge as inline HTML, drawn from the gauge marks file.

Code: `MethylPhys_Interface/report_v3.py` line 51 (38 lines) · arguments: A, lo, hi, w, label, instrument, top · calls: array run 1, bam run 2 · files named: `CANON/foundation.json`, `Runtime Matrices/Gauge/cell_gauge_marks_v1.json`, `drawn from Runtime Matrices/Gauge/cell_gauge_marks_v1.json`

`report_v3.py::_H`

The entropy of a rate in bits, for the formula self-test.

Code: `MethylPhys_Interface/report_v3.py` line 90 (3 lines) · arguments: b · calls: array run 3, bam run 3

`report_v3.py::_load`

Loads a chain file, or nothing if it is missing.

Code: `MethylPhys_Interface/report_v3.py` line 95 (3 lines) · arguments: rel · calls: array run 3, bam run 3 · constants read: `CHAIN`

`report_v3.py::red_flags`

The red flags, from the bundle only.

Code: `MethylPhys_Interface/report_v3.py` line 101 (26 lines) · arguments: o · calls: array run 1, bam run 1

`report_v3.py::safeguards`

The safeguards listed above.

Code: `MethylPhys_Interface/report_v3.py` line 129 (44 lines) · arguments: o, prose · calls: array run 1, bam run 1 · constants read: `CLAIM_SCAN` · files named: `Runtime Matrices/Met_A_Floors/metA_floors_v1_3.json`, `Runtime Matrices/Met_A_Floors/neutrophil_reference_v1_2.json`, `Runtime Matrices/Met_A_Floors/noise_gate_EPIC_v1.json`

report_v3.py::troubleshooting

For each stop on this run, what it usually means and what to do.

Code: MethylPhys_Interface/report_v3.py line 175 (17 lines) · arguments: o · calls: array run 1, bam run 1

report_v3.py::toolkit_table

The development stages and whether each ran on this sample.

Code: MethylPhys_Interface/report_v3.py line 194 (12 lines) · arguments: o · calls: array run 1, bam run 1 · constants read: NOT_BUILT, TOOLKIT_STAGES

report_v3.py::_table

A plain HTML table.

Code: MethylPhys_Interface/report_v3.py line 208 (4 lines) · arguments: rows, head · calls: array run 7, bam run 7

report_v3.py::build

The technical record: intake, Stage 1, the mix, Met-A, the tare, the noise gate, the C-Score, IAM-A, the red flags, the safeguards, troubleshooting and the toolkit table, the run-it-yourself command, and a copy of the bundle inside the page. Returns the page's path, size and any refusals; the bundle file itself is written by run_sample.py.

Code: MethylPhys_Interface/report_v3.py line 214 (83 lines) · arguments: o, out, sid · calls: array run 1, bam run 1 · constants read: FLAG_ORDER, STAGES_2B · files named: cell_gauge_marks_v1.json

report_figs_v4.py::sci

A number in scientific notation for a figure.

Code: MethylPhys_Interface/report_figs_v4.py line 13 (2 lines) · arguments: v · calls: not called on the traced runs

report_figs_v4.py::_bar

Draws one gauge bar: the gradient from the floor through healthy to the ceiling, the band, the needle and the ticks.

Code: MethylPhys_Interface/report_figs_v4.py line 15 (13 lines) · arguments: ax, lo_v, hi_v, band, A, fl, cl, y0, h, grad, ticks, mid · calls: array run 2, bam run 2

report_figs_v4.py::gauge_png

The floor-to-ceiling gauge for Met-A or IAM-A, with the magnified strip when healthy sits near the floor.

Code: MethylPhys_Interface/report_figs_v4.py line 29 (20 lines) · arguments: out, name, A, floor, ceiling, band, subject, state, unit · calls: array run 1, bam run 1

report_figs_v4.py::cscore_png

The C-Score bar, scattered to clustered; the scale stretches to hold the reading.

Code: MethylPhys_Interface/report_figs_v4.py line 50 (10 lines) · arguments: out, C, band_hi, band_lo, subject, state · calls: array run 1

report_figs_v4.py::atlas_sky_png

The healthy reference of the sample's cell from IAM-Atlas v2.1, at the same sites, in the same order and on the same pixels as the sample's C-Score sky, so the two line up site for site.

Code: `MethylPhys_Interface/report_figs_v4.py` line 61 (40 lines) · arguments: out, ids, cell, atlas_dir, nest_pix2ang, title · calls: array run 1 · files named: `Planck_Parchment_RGB.txt`, `import pyarrow.parquet`, `sites.parquet`

`report_figs_v4.py::changes_sky_png`

The sample laid over its healthy reference: the healthy cell in grays, color only where the sample departs by more than a threshold of its own spread. Single colored pixels are ordinary; colored patches are changes that run along the genome, which is what the C-Score measures.

Code: `MethylPhys_Interface/report_figs_v4.py` line 102 (43 lines) · arguments: out, z, cell, atlas_dir, nest_pix2ang, readings, thr, title · calls: array run 1 · files named: `import pyarrow.parquet`, `sites.parquet`

`report_figs_v4.py::changes_sky_png.pmean`

The mean of the values falling in each pixel.

Code: `MethylPhys_Interface/report_figs_v4.py` line 119 (2 lines) · arguments: x · calls: array run 2

`report_pdf_v4.py::P`

A paragraph for the PDF, keeping only the tags the PDF library understands.

Code: `MethylPhys_Interface/report_pdf_v4.py` line 25 (4 lines) · arguments: t, st · calls: array run 90, bam run 49 · constants read: S

`report_pdf_v4.py::_img`

A figure scaled to a width, or nothing if missing.

Code: `MethylPhys_Interface/report_pdf_v4.py` line 30 (4 lines) · arguments: path, w · calls: array run 8, bam run 6

`report_pdf_v4.py::_head`

The PDF's heading block, with the research-use line and how to read the commissioning status.

Code: `MethylPhys_Interface/report_pdf_v4.py` line 35 (8 lines) · arguments: o, sid, when, view · calls: array run 2, bam run 2

`report_pdf_v4.py::_tbl`

A PDF table.

Code: `MethylPhys_Interface/report_pdf_v4.py` line 44 (5 lines) · arguments: rows, widths, head · calls: array run 3, bam run 2

`report_pdf_v4.py::build_pdfs`

The researcher and plain PDFs, from the same rows and pictures as the page.

Code: `MethylPhys_Interface/report_pdf_v4.py` line 50 (50 lines) · arguments: o, F, out, sid, when · calls: array run 1, bam run 1

`report_pdf_v4.py::build_pdfs.doc`

Writes one PDF.

Code: `MethylPhys_Interface/report_pdf_v4.py` line 53 (3 lines) · arguments: path, story · calls: array run 2, bam run 2

15. The development readings: `dev_stages.py`, `dev_comp_methods.py`, `deconv_v2.py`, `sky_statistics.py`

What they are for. Readings under development, run only when a `--dev-*` flag asks for them. Each writes its own record under `bundle["development"]`, labeled "DEVELOPMENT - not commissioned", and none changes the reading, the gauge or the tare. They are in the chain so a development result is made by the same code a researcher can run. The one exception is `dev_stages.selftare_map` (section 5), which has been Stage T step 1 on every reading since 2026-10-04 and still lives in this file.

Labels. Direction, brightness and the per-cell B reading follow Met-A's physics; the composition methods and the sky statistics are standard methods (deconvolution, the angular power spectrum of a sphere with a shuffled null).

`dev_stages.py::_H`

The entropy of a beta value in bits.

Code: `dev_stages.py` line 32 (3 lines) · arguments: b · calls: not called on the traced runs

`dev_stages.py::_bc`

Loads the blood composition matrix once.

Code: `dev_stages.py` line 44 (3 lines) · arguments: none · calls: not called on the traced runs · constants read: `RM`, `_C` · files named: `blood_composition_EPIC_v1.json`

`dev_stages.py::_rec`

A development record with its label.

Code: `dev_stages.py` line 49 (2 lines) · arguments: stage · calls: not called on the traced runs · constants read: `DEV_LABEL`

`dev_stages.py::selftare_ii`

The development form of self-tare II: Met-A re-read on the mapped values beside the reading. Kept for its earlier records; the one command no longer calls it, because step 1 now runs on every reading.

Code: `dev_stages.py` line 77 (9 lines) · arguments: beta, specimen · calls: not called on the traced runs

`dev_stages.py::site_uncertainty`

This array's own per-site uncertainty: the SD of β against the reference value over its fixed sites, per design and state.

Code: `dev_stages.py` line 89 (8 lines) · arguments: beta, st · calls: not called on the traced runs · files named: `dev_selftare_typeII_EPIC_v1.json`

`dev_stages.py::s_for`

The per-site uncertainty for a list of sites, by design and state (low when the reference value is below 0.5).

Code: `dev_stages.py` line 99 (6 lines) · arguments: sites, ref_values, su, st · calls: not called on the traced runs · files named: `dev_selftare_typeII_EPIC_v1.json`

`dev_stages.py::direction`

D = mean over the identity sites of |r – 0.5| – |β – 0.5|: above 0, the sites have moved toward 0.5, toward disorder; below 0, away from it.

Code: `dev_stages.py` line 108 (6 lines) · arguments: beta, ref, sites · calls: not called on the traced runs

`dev_stages.py::direction_record`

The direction record at the neutrophil identity sites against the cell's own reference (the expectation in whole blood), tared on the same-run references' D.

Code: `dev_stages.py` line 116 (15 lines) · arguments: beta, specimen, comp, refs_D · calls: not called on the traced runs

dev_stages.py::trace_cell

In an isolated-neutrophil specimen, another blood group's contamination, called above 3 × this array's own standard error.

Code: `dev_stages.py` line 134 (15 lines) · arguments: beta, su, line · calls: not called on the traced runs · files named: `dev_selftare_typeII_EPIC_v1.json`

dev_stages.py::foreign_cell

In whole blood, a non-blood template solved beside the 8 blood groups, called above 3 × this array's own standard error.

Code: `dev_stages.py` line 152 (22 lines) · arguments: beta, template, su, line · calls: not called on the traced runs · files named: `dev_foreign_placenta_EPIC_v1.json`, `dev_selftare_typeII_EPIC_v1.json`

dev_stages.py::brightness

A 95 % interval on Met-A by Monte Carlo from this array's per-site uncertainty.

Code: `dev_stages.py` line 177 (17 lines) · arguments: beta, specimen, comp, n_draw, su, seed · calls: not called on the traced runs · files named: `dev_selftare_typeII_EPIC_v1.json`

dev_stages.py::_mapping

Loads the CpG-to-HEALPix map of the development sky (NSIDE 128).

Code: `dev_stages.py` line 197 (6 lines) · arguments: none · calls: not called on the traced runs · constants read: `RM`, `_C` · files named: `iamatlas_cpg_to_healpix_nside128.npz`

dev_stages.py::residual_z

The development sky's per-site departure: z = (β – Σ f_g μ_g) / √(Σ f_g² (sd² + s_bp²) + 0.02²), with s_bp the atlas's between-person SD.

Code: `dev_stages.py` line 205 (6 lines) · arguments: beta, fractions, AP · calls: not called on the traced runs · constants read: `PARENTS`

dev_stages.py::_pixmap

The mean departure in each pixel.

Code: `dev_stages.py` line 213 (3 lines) · arguments: zs, pix, npix · calls: not called on the traced runs

dev_stages.py::block_shuffle_order

A permutation that moves whole runs of consecutive sites within each chromosome: the null keeps short-range structure along the genome and breaks where it sits.

Code: `dev_stages.py` line 218 (7 lines) · arguments: chr_keys, run, rng · calls: not called on the traced runs · constants read: `SKY_RUN`

dev_stages.py::sky_from_z

The development sky's band powers against the block-shuffle null and the look-elsewhere statistic by simulation.

Code: `dev_stages.py` line 227 (14 lines) · arguments: z, n_null, n_lee, seed, free_null · calls: not called on the traced runs · constants read: `SKY_LEE_N`, `SKY_NULL_N`, `SKY_RUN`

dev_stages.py::sky_from_z.shuf

The band powers of one block-shuffled sky.

Code: `dev_stages.py` line 232 (2 lines) · arguments: none · calls: not called on the traced runs

dev_stages.py::_has_cols

Whether a parquet file holds the named columns.

Code: `dev_stages.py` line 243 (4 lines) · arguments: path, cols · calls: not called on the traced runs · files named: `import pyarrow.parquet`

dev_stages.py::load_atlas_parents

Loads the atlas v2 parent blood cells' means, SDs and between-person SDs.

Code: `dev_stages.py` line 249 (8 lines) · arguments: atlas_parquet · calls: not called on the traced runs · constants read: `PARENTS`, `_C`

dev_stages.py::sky

The development residual sky of one whole blood, with its null.

Code: `dev_stages.py` line 259 (7 lines) · arguments: beta, fractions, atlas_parquet · calls: not called on the traced runs

dev_stages.py::_atlas_e

Builds the atlas_e solver once on the 12 array-measured circulating cells of atlas v2, with the neutrophil identity sites removed.

Code: `dev_stages.py` line 273 (9 lines) · arguments: atlas_parquet · calls: not called on the traced runs · constants read: `ATLAS_E_CELLS`, `_C`

dev_stages.py::atlas_e

The atlas_e composition, summed into the 8 blood groups.

Code: `dev_stages.py` line 284 (5 lines) · arguments: beta, atlas_parquet · calls: not called on the traced runs · constants read: `BLOOD8`

dev_stages.py::nilc_e

The NILC-e composition on the atlas_e templates, with each share's noise.

Code: `dev_stages.py` line 291 (5 lines) · arguments: beta, atlas_parquet · calls: not called on the traced runs · constants read: `BLOOD8`

dev_stages.py::percell_b

A B-cell Met-A against the expectation for this blood's mix, on the development B-cell reference.

Code: `dev_stages.py` line 298 (13 lines) · arguments: beta, fractions, atlas_parquet · calls: not called on the traced runs · constants read: `PARENTS`, `RM` · files named: `metA_floors_v1_2_ALLCELLS_development.json`

dev_stages.py::epicv2_calibrate

An EPIC v2 IDAT pair through SeSAMe (an R package, run in its own environment) to β values on EPIC v1 probe names, replicate probes of one CpG averaged and background-masked probes dropped. Development only: the chain's references are EPIC v1.

Code: `dev_stages.py` line 317 (19 lines) · arguments: grn, red, rscript, prep · calls: not called on the traced runs · constants read: `SESAME_R` · files named: `betas.csv`

dev_comp_methods.py::load_atlas

Loads an atlas parquet file indexed by site.

Code: `dev_comp_methods.py` line 46 (4 lines) · arguments: path · calls: not called on the traced runs

dev_comp_methods.py::atlas_cells

The cell names in an atlas table.

Code: `dev_comp_methods.py` line 52 (2 lines) · arguments: A · calls: not called on the traced runs

dev_comp_methods.py::subset_atlas

The columns of the named cells.

Code: `dev_comp_methods.py` line 56 (3 lines) · arguments: A, keep · calls: not called on the traced runs

dev_comp_methods.py::build_deconv

An atlas v2 solver on the named cells, with the neutrophil identity sites removed from its pool.

Code: `dev_comp_methods.py` line 61 (4 lines) · arguments: A, keep, exclude_sites · calls: not called on the traced runs · constants read: `POOL1`, `SOLVER`

dev_comp_methods.py::twin_merge

The atlas's own twin rule: a cell with no array measurement whose marker profile correlates with an array-measured cell above the threshold is that cell on another platform, and the two merge.

Code: `dev_comp_methods.py` line 67 (28 lines) · arguments: A, D, roster, twin_tab, thr, candidates · calls: not called on the traced runs · files named: `records/10_twin_test_sample_level.csv`, `twin_family_thresholds_v1.json`

dev_comp_methods.py::mixture_twin

A cell whose template is reproduced by a non-negative, sum-to-1 mix of the other cells, above the twin correlation, cannot be told apart from that mix in a specimen.

Code: `dev_comp_methods.py` line 97 (18 lines) · arguments: D, candidates, r_thr · calls: not called on the traced runs

dev_comp_methods.py::crlb

The Cramér–Rao bound on each share's SD for a solver's design at its markers.

Code: `dev_comp_methods.py` line 117 (10 lines) · arguments: D, f, sigma · calls: not called on the traced runs

dev_comp_methods.py::NNLS8.__init__

Stage A's solver as a class: the 963 markers and 8 groups.

Code: `dev_comp_methods.py` line 130 (2 lines) · arguments: self, bc · calls: not called on the traced runs

dev_comp_methods.py::NNLS8.deconvolve

Stage A's shares, for comparison with the other methods.

Code: `dev_comp_methods.py` line 132 (4 lines) · arguments: self, beta · calls: not called on the traced runs

`dev_comp_methods.py::ledoit_wolf`

Analytic Ledoit–Wolf shrinkage of a covariance toward a multiple of the identity; no parameter to choose.

Code: `dev_comp_methods.py` line 138 (7 lines) · arguments: X · calls: not called on the traced runs

`dev_comp_methods.py::NILC.__init__`

The NILC solver: sites, cells, means, variances and the noise model.

Code: `dev_comp_methods.py` line 150 (4 lines) · arguments: self, sites, cells, mu, v, cov, sigma, tech_var, tech_cov, offset · calls: not called on the traced runs

`dev_comp_methods.py::NILC._solve`

One weighted, constrained solve.

Code: `dev_comp_methods.py` line 154 (11 lines) · arguments: self, m, f · calls: not called on the traced runs

`dev_comp_methods.py::NILC.deconvolve`

The NILC shares in two passes.

Code: `dev_comp_methods.py` line 165 (9 lines) · arguments: self, beta, passes · calls: not called on the traced runs

`dev_comp_methods.py::to_groups`

Sums atlas cells into the 8 blood groups.

Code: `dev_comp_methods.py` line 176 (7 lines) · arguments: fr · calls: not called on the traced runs · constants read: `SUBGROUP`

`dev_comp_methods.py::met_a`

Whole-blood Met-A with another method's shares and profiles, for comparison with Stage M.

Code: `dev_comp_methods.py` line 185 (8 lines) · arguments: beta, fractions, profiles, ns · calls: not called on the traced runs · constants read: `H`

`dev_comp_methods.py::ceiling_flag`

The entropy-ceiling flag for that comparison reading.

Code: `dev_comp_methods.py` line 195 (16 lines) · arguments: beta, profiles, fractions, ns · calls: not called on the traced runs

`dev_comp_methods.py::read_beta`

Reads one β file.

Code: `dev_comp_methods.py` line 213 (3 lines) · arguments: path · calls: not called on the traced runs

`deconv_v2.py::DeconvV2.__init__`

The atlas v2 solver: loci measured by every modeled cell; for each cell, markers chosen against its nearest other cell (margin at least 0.20, the top 200 by margin over pooled SD); variance v = μ_sd² + donor_sd².

Code: `deconv_v2.py` line 11 (34 lines) · arguments: self, atlas_parquet, pooled_one_sample, margin, top, min_markers, sigma, drop, markers, k_near, per_pair, pair_margin, A · calls: not called on the traced runs

deconv_v2.py::DeconvV2._solve

One joint solve with f ≥ 0 and Σ f = 1, weights 1 / (Σ_c f_c² v_ci + σ²) iterated 5 times.

Code: deconv_v2.py line 46 (7 lines) · arguments: self, y, M, V · calls: not called on the traced runs

deconv_v2.py::DeconvV2.deconvolve

The shares, and each cell's presence by 200 bootstrap resamples of the markers.

Code: deconv_v2.py line 54 (20 lines) · arguments: self, beta, n_boot, seed, present_floor, affine · calls: not called on the traced runs

sky_statistics.py::bandpowers

The mean power in each of six bands of the angular spectrum.

Code: sky_statistics.py line 30 (3 lines) · arguments: cl, bands · calls: not called on the traced runs · constants read: BANDS

sky_statistics.py::apodised_mask

A pixel weight that rises smoothly from 0 at the edge of the measured footprint to 1 a set angle inside it, so a sharp edge does not spread power between multipoles.

Code: sky_statistics.py line 35 (20 lines) · arguments: good, apod_deg, taper · calls: not called on the traced runs

sky_statistics.py::_weights

Keeps the last mask's weights.

Code: sky_statistics.py line 57 (5 lines) · arguments: good, apod_deg, taper · calls: not called on the traced runs · constants read: _W

sky_statistics.py::masked_spectrum

The pseudo angular power spectrum of a sky with masked pixels, with a hard or apodized mask.

Code: sky_statistics.py line 64 (13 lines) · arguments: pix, lmax, mask, apod_deg, taper · calls: not called on the traced runs · constants read: MASKS

sky_statistics.py::shuffled_null

The band powers of skies made by shuffling the sky's own measured pixel values among its measured pixels: the mask, the count and the distribution are kept, and only the arrangement is destroyed, so a difference from this null is spatial structure.

Code: sky_statistics.py line 79 (16 lines) · arguments: pix, n_perm, lmax, rng, bands, mask, apod_deg, taper · calls: not called on the traced runs · constants read: BANDS, MASKS

sky_statistics.py::sky_spectrum_record

One sky's spectrum, band powers, measured fraction, share of pixels with |z| above 2, and its null.

Code: sky_statistics.py line 97 (11 lines) · arguments: pix, n_perm, lmax, rng, mask, apod_deg, taper · calls: not called on the traced runs · constants read: BANDS

Part 2. Instrument diagnostics: before a run, reading the report, when the chain stops

Source: `manual/OM_v4_chain.md`

Operations Manual: chain v4 (operator chapter)

The procedure (install, the one-command check, the commands, what is declined, outputs, frozen inputs) is the SOP, printed first in this manual. This chapter is what an operator needs around it: before a run, reading the report, and what to do when the chain stops.

Before a run

1. Run the release check once per installation and after every update: `python3 release_check_v4.py`. Every check must pass.
2. Arrays: the Illumina manifest must be in `$HOME/.methylprep_manifest_files/` (methylprep fetches it on first use). Read the first array of a batch alone, so the download is not raced.
3. Same-run references (arrays): read three or more healthy arrays from the same run first, each on its own. Put their Met-A reading (`A`) and C-score (`C`) from each report's bundle in a CSV with columns `id,A,C`, and pass it with `--ref-table` when reading the samples of that run. The tare divides by the references' median, so the laboratory's own habits cancel out. Nothing is fitted.
4. Sequencing: give the cell type with `--seq-cell`, and for a BAM the exact reference FASTA it was aligned to (`--ref-fasta`, indexed).
5. Declare the specimen exactly (`--specimen "whole blood"`, `"isolated neutrophils"`, `"buffy coat"` or `"constructed DNA mixture"`). Anything else is refused at intake with its reason.

Reading the report

Each run writes one HTML page (a Researcher view, opened first, and a Plain view), the same report as two PDFs, the bundle (JSON) and one row in the evidence ledger. The report reads top to bottom:

- **At a glance.** One line per instrument: its reading, its state, and whether the reading is inside that instrument's commissioned scope.
- **Every gauge has three marks:** 0, the physical floor; 1, healthy; and the saturation end, where every identity site is fully scrambled (Met-A 3.03; IAM-A 0.5 / ε_pred(L), the sample's own). The marks are derived, not fitted.
- **States.** Met-A, tared: Normal 0.95 to 1.05. The C-score: above its healthy band when above 1.152 (one-sided). IAM-A: Normal 0.7787 to 1.0551. Where these bands mean disease is for cancer researchers to find; the report prints readings and states, not diagnoses.
- **Commissioned scope.** A reading outside what an instrument was commissioned on is still shown, with the line *This score is preliminary until [instrument] is officially commissioned on [specimen or cell]*. Met-A is commissioned on isolated neutrophils, the C-score on whole blood, the IAM-Deconvolver on neutrophil, monocyte, CD4 T and CD8 T shares, IAM-A on healthy CD8 T cells.
- **Cell shares.** Each cell's share with its 95 % interval, read against IAM-Atlas v2.1. Before mixing anything, the tool checks the sample can be read; it refuses rather than report a wrong share.
- **The tare.** When `--ref-table` is given, the report prints the references' median and spread, the tared reading, and the smallest change this sample could have shown.
- **The noise gate.** On a noisy array Met-A's state is withheld and no gauge is drawn; the report says so.
- **The sky.** The sample laid over healthy neutrophils from IAM-Atlas v2.1 on one HEALPix sphere, every site in its genome place. Grey is where the sample matches healthy; color marks only departures beyond twice the

sample's own spread. Scattered color is what healthy looks like; streaks along the genome are what the C-score calls clustered.

- **IAM-A** reads a sequencing sample against itself: the error with which its own cells hold their marks, against the healthy line of its declared cell type. If the sample's shape is outside that cell type's healthy shape, or too few molecules qualify, IAM-A declines and says why.
- **Technical record.** The chain's commit, every input's checksum, every frozen file read, and every caution (for example, Stage 0 did not run because the sample entered as a table of values).

When the chain stops

exit	printed	meaning	what to do
0	the report	read, or declined with a reason printed in the report (specimen, platform, sample shape)	read the report
1	an argument message	a command-line input is missing or malformed (for example a reference table without its columns)	correct the command
2	`QUARANTINE` and its code	Stage 0 failed the file (corrupt or truncated IDAT, missing channel, array type mismatch, low detection)	nothing is scored; check the file, rerun from the scanner output
3	`ENVIRONMENT_MISSING_MANIFEST`	the Illumina manifest could not be fetched	put the manifest in place and rerun; the array was not judged

An EPIC v2 or 450K array is refused with one sentence (`PLATFORM_REFUSED`); the chain holds no Met-A reference for those platforms.

Reproducing the commissioning

The commissioning tests and the headline results rebuild from public data with one command each: `Biological_Physics/MethylPhys/reproduce/README.md` at the top of the repository.

Part 3. The chain, step by step

Source: doors/CHAIN_SEQUENCE.md

The chain, step by step - derived from the code

Generated by chain/build_chain_sequence.py from the AST of run_sample.py, conductor_v3.py and stage_q_iam_a.py. Do not edit by hand: re-run the generator. Every step below is a call the code actually makes, in the order it makes it. Chain v3 is the only engine; the class-floor engine (v2) was retired on 2026-10-03 and is archived privately.

The live path (chain v4) - 32 steps, in order

#	step	implemented in	what it does
1	specimen_refusal	stage_0_intake.py	None when intake accepts the specimen (ACCEPTED_SPECIMENS); otherwise the plain refusal text naming the specimen (decision L)
2	step_0_1_idat_arrival	stage_0_intake.py	SOP section 0.1_idat_arrival
3	step_0_2_manifest_creation	stage_0_intake.py	SOP section 0.2_manifest_creation
4	step_0_3_integrity_hash	stage_0_intake.py	SOP section 0.3_integrity_hash
5	step_0_4_control_probe_validation	stage_0_intake.py	SOP section 0.4_control_probe_validation
6	step_0_5_detection_pvalue_qc	stage_0_intake.py	SOP section 0.5_detection_pvalue_qc
7	step_0_6_bead_count_qc	stage_0_intake.py	SOP section 0.6_bead_count_qc
8	step_0_7_call_rate	stage_0_intake.py	SOP section 0.7_call_rate
9	step_0_7b_platform_coverage	stage_0_intake.py	SOP section 0.7b_platform_coverage
10	step_0_8_sex_check	stage_0_intake.py	SOP section 0.8_sex_check
11	step_0_9_decision_gate	stage_0_intake.py	SOP section 0.9_decision_gate
12	Stage 1 - IDAT calibration	stage_1_idat_calibration.py	Stage 1 - IDAT calibration to beta (SOP Stage 1, steps 1.1-1.2 + 1.5) _called by run_sample.py when given --grn/--red; skipped when given --betas_
13	platform_refusal	conductor_v3.py	None when the specimen is EPIC v1; otherwise the refusal text
14	stage_t_selftare_ii	conductor_v3.py	Stage T step 1, self-tare II (adopted 2026-10-04, DEV-SELFTARE-02): per probe design, beta' = Lr + (beta - L)(Ur - Lr)/(U - L) from this
15	stage_m_isolated	conductor_v3.py	Isolated / sorted neutrophils against their healthy reference. The healthy-reference state is kept as state_own_floor; the reading's state is
16	stage_a_composition	conductor_v3.py	EPIC blood composition (blood_composition_EPIC_v1): 8 groups from 91 purified-cell EPIC arrays (GSE110554, GSE167998); markers exclude the neutrophil sites;
17	stage_m_blood	conductor_v3.py	Whole blood: Met-A = mean H(beta) / mean H(e) at the neutrophil sites, e = sum_g f_g mu_g (EPIC purified group profiles)

#	step	implemented in	what it does
18	`noise_index`	`conductor_v3.py`	N = mean H(beta) over the noise sites measured on this array; None when fewer than MIN_NOISE_FRACTION are measured _stage 9: the array's noise index N over the noise sites_
19	`stage_t_tare`	`conductor_v3.py`	Same-run tare against >= MIN_REFS healthy references of the same specimen type run the same way (same slide, else same batch)
20	`noise_gate`	`conductor_v3.py`	Runtime Matrices/Met_A_Floors/noise_gate_EPIC_v1.json: N_max = top of the reference arrays' noise range (DEV-NOISE-01) _stage 9: N_max read here; run_neutrophil withholds the gauge state when N > N_max and the reading is untared_
21	`stage_mc_cscore`	`conductor_v3.py`	Met-A C-score (stage 6): clustering of the residual z map in genomic order - variance of the means of blocks of clustering_block
22	`stage_t_cscore`	`conductor_v3.py`	Same-run tare of the Met-A C-score, the same rule as the median tare of A (DEV-CSCORE-TARE-01): C_rel = C / median(C of >= MIN_REFS
23	`stage_v4.stage_d21`	`stage_v4.py`	IAM-Deconvolver: cell shares against IAM-Atlas v2.1 _array input_
24	`stage_v4.cscore_sky`	`stage_v4.py`	z: the C-Score's per-site departure map (pd.Series by site, in the C-Score's genomic order). Writes a Mollweide view; returns a summary _array input; a picture, nothing scored_
25	`stage_q0_intake.intake`	`stage_q0_intake.py`	Stage Q0 on one .pat(.gz). alignment_qc: dict (or JSON path) from the alignment step with 'conversion_rate' and/or _runs when run_sample.py is given --pat, before Stage Q_
26	`stage_v4.stage_q2`	`stage_v4.py`	IAM-A: count, sample-shape check, reading against the sample's own loose share on band v1 _runs when run_sample.py is given --pat or --bam (the retired position reading only with --dev-iama-position)_
27	`stage_q_iam_a.pat_site_table`	`stage_q_iam_a.py`	Per-site table (pos, opp_A, err_A, opp_B, err_B) from one .pat(.gz) file, pipeline loyfer_pat_v1. pos = 'chrom:CpG index' _development: --site-table, or --pat with --dev-iama-position_
28	`stage_q_iam_a.read`	`stage_q_iam_a.py`	site_table: per-site table (see module doc). cell: e.g. 'neutrophils'. pipeline: the read-level pipeline that produced the table _development: --site-table, or --pat with --dev-iama-position_
29	`stage_q_iam_a.cscore`	`stage_q_iam_a.py`	IAM-A C-score for the pooled table (opp = opp_A + opp_B) and for each half. Returns {C, n_blocks, se_null, halves: {...}} _development: --site-table, or --pat with --dev-iama-position_

#	step	implemented in	what it does
30	`serial_mode.check_same_person`	`serial_mode.py`	Refuse unless patient hash, array type and pipeline match. Returns (ok, reason) _stage 12b: runs when run_sample.py is given --prior-betas and --prior-bundle_
31	`serial_mode.delta_sky`	`serial_mode.py`	beta_now - beta_prior per address on the addresses both draws measured. No expectation, no sigma, no composition _stage 12b: runs when run_sample.py is given --prior-betas and --prior-bundle_
32	`Report`	`MethylPhys_Interface/report_v4.py`	one self-contained HTML page plus the JSON bundle

Stage modules in chain/ that the live path does not call

module	what it implements	status
`stage_2c_trace_detection.py`	Stage 2c - is there any evidence of a trace class in this specimen?	module present; neither run_sample.py nor conductor_v3.py calls it (toolkit, chain/TOOLKIT.md)
`stage_4_6_patient_cmb.py`	Stage 4.6 — the patient's sky (PROC-CMB-01, 2026-09-21). Replaces the retired patient_brightness_comparison.py	module present; neither run_sample.py nor conductor_v3.py calls it (toolkit, chain/TOOLKIT.md)
`stage_a_composition_v2.py`	Stage A - composition, atlas v2 solver (deconv_v2). Not called by chain v3 (conductor_v3.stage_a_composition uses blood_composition_EPIC_v1.json). Frozen settings (commissioned as PROC-DECONV-V2-01; specimen cell sets 2026-09-30)	module present; neither run_sample.py nor conductor_v3.py calls it (toolkit, chain/TOOLKIT.md)
`stage_atlas_commission.py`	Stage A21 - atlas v2.1 commissioning scorers, as AMENDED 2026-10-16 before the atlas exists (author: "Seal the amendment")	module present; neither run_sample.py nor conductor_v3.py calls it (toolkit, chain/TOOLKIT.md)
`stage_atlas_u1.py`	Atlas v2.1 use test U1 (reference profiles), scored ONCE by the sealed and amended bar (chain Stage A21), written 2026-10-16 before any	module present; neither run_sample.py nor conductor_v3.py calls it (toolkit, chain/TOOLKIT.md)
`stage_atlas_u1_planted.py`	Planted test of score_u1_v21.py (synthetic first): fake blocks with known draws, arrays drawn from the predictive distribution (must pass),	module present; neither run_sample.py nor conductor_v3.py calls it (toolkit, chain/TOOLKIT.md)
`stage_atlas_u2.py`	Atlas v2.1 U2a, blind sites of the four sealed tissue tests (sealed_commission_v21.json U2; bar B2 of amendment_v21b.json; chain Stage A21	module present; neither run_sample.py nor conductor_v3.py calls it (toolkit, chain/TOOLKIT.md)
`stage_atlas_u3.py`	Atlas v2.1 U3, blood composition (sealed bar B3, amendment_v21b.json; scorer chain Stage A21 u3_neutrophil). Written 2026-10-16 before any	module present; neither run_sample.py nor conductor_v3.py calls it (toolkit, chain/TOOLKIT.md)
`stage_deconv_v21.py`	Stage D21 - the deconvolution tool (IAM-Deconvolver) on IAM-Atlas v2.1. Written 2026-10-16; COMMISSIONED 2026-10-17 for the shares of	module present; neither run_sample.py nor conductor_v3.py calls it (toolkit, chain/TOOLKIT.md)
`stage_temperature.py`	Chain stage TL: the temperature law on single molecules (PRED-TEMPERATURE-01, PHY-057). Written 2026-10-17, before any read is opened	module present; neither run_sample.py nor conductor_v3.py calls it (toolkit, chain/TOOLKIT.md)

Behind development flags (DEVELOPMENT - not commissioned; never part of a reading)

flag	step	implemented in	what it does
`--dev-direction`	`dev_stages.direction_record`	`dev_stages.py`	Signed move toward or away from 0.5 at the neutrophil identity sites against the cell's own reference, tared on same-run references (DEV-DIRECTION-02)
`--dev-trace`	`dev_stages.trace_cell`	`dev_stages.py`	3b: another blood group in an isolated-neutrophil specimen, called above 3 x this array's own standard error (DEV-TOOLKIT-ADDED-02)
`--dev-foreign`	`dev_stages.foreign_cell`	`dev_stages.py`	3c: a non-blood template in whole blood by NNLS with the blood groups, called above 3 x this array's own standard error (DEV-TOOLKIT-ADDED-02)
`--dev-brightness`	`dev_stages.brightness`	`dev_stages.py`	11b: 95 % interval on Met-A from the v3 per-site uncertainty (this array's own fixed-site noise), by Monte Carlo (DEV-TOOLKIT-ADDED-02)
`--dev-nilc`	`dev_stages.nilc_e`	`dev_stages.py`	Stage 4 NILC-e composition on the atlas_e templates (DEV-NILC-01 method), 8-group sums and the noise per fraction
`--dev-atlas-e`	`dev_stages.atlas_e`	`dev_stages.py`	Stage 3 atlas_e composition (12 array-measured circulating atlas v2 cells, DEV-ATLAS-EPIC-02 method), 8-group sums
`--dev-percell-b`	`dev_stages.percell_b`	`dev_stages.py`	Stage 5 B-cell Met-A against the composition-matched expectation on the development B-cell floor (DEV-PERCELL-01 method)
`--dev-sky`	`dev_stages.sky`	`dev_stages.py`	Residual sky with the within-chromosome block-shuffle null and the look-elsewhere statistic by simulation (DEV-SKY-02)
`--dev-epic-v2`	`dev_stages.epicv2_calibrate`	`dev_stages.py`	EPIC v2 IDAT pair -> beta Series on EPIC v1 probe names (replicate v2 probes of one CpG averaged). SeSAMe openSesame(prep), pOOBAH-masked

The toolkit (chain/TOOLKIT.md) - built; each stage wired only after its check passed

stage (SOP 2b)	name	module	status
3	Atlas deconvolution	`chain/deconv_v2.py`	toolkit: not yet wired into chain v3
3b	Trace-cell detection	`chain/dev_stages.py`	toolkit: not yet wired into chain v3
3c	Foreign-cell detection	`chain/dev_stages.py`	toolkit: not yet wired into chain v3
4	NILC component separation	`chain/dev_stages.py`	toolkit: not yet wired into chain v3
4	NILC helper: lineage splitter	`chain/Lineage_Splitter/lineage_splitter.py`	toolkit: not yet wired into chain v3
10	Directional decomposition	`chain/dev_stages.py`	toolkit: not yet wired into chain v3
11	Sky map	`chain/dev_stages.py`	toolkit: not yet wired into chain v3
11b	Surface brightness	`chain/dev_stages.py`	toolkit: not yet wired into chain v3
12	Sky statistics	`chain/sky_statistics.py`	toolkit: not yet wired into chain v3
12b	Difference map	`chain/serial_mode.py`	wired into chain v3 (optional input; see chain/TOOLKIT.md)

stage (SOP 2b)	name	module	status
5	Met-A per cell (B cells)	`chain/dev_stages.py`	toolkit: not yet wired into chain v3
8	Self-tare on type II fixed sites	`chain/dev_stages.py`	wired into chain v3: always runs (Stage T step 1); see chain/TOOLKIT.md
1	EPIC v2 calibration	`chain/dev_stages.py`	toolkit: not yet wired into chain v3

Sources

manual/OM_v4_decoder.md sha256 936be2e703de

manual/OM_v4_chain.md sha256 e29d10b516eb

doors/CHAIN_SEQUENCE.md sha256 769fcbab39e1